Parallel MPPI With Gradient-Velocity Modulated SDF Cost for High-Performance Real-Time Dynamic Obstacle Avoidance by Robot Manipulators

Lelai Zhou, *Member, IEEE*, Zhengmao Li, Yibin Li, *Member, IEEE*, and Shaoping Bai, *Senior Member, IEEE*

***Abstract*—Real-time motion planning in dynamic environments presents a significant challenge for robotic manipulators. This article introduces an innovative parallel model predictive path integral (MPPI) algorithm enabling the robot to navigate swiftly and safely in such environments. Unlike the conventional MPPI methods that rely on a single sequence of Gaussian means for trajectory sampling, the proposed parallel MPPI (PMPPI) concurrently runs multiple planners with different strategies and adaptively integrates planned paths based on the current state, leveraging the advantages of different strategies and greatly improving the MPPI's exploration capability. Moreover, a gradient-velocity modulated signed distance field (SDF) cost function that dynamically adjusts costs based on the robot's velocity and the SDF gradient is defined, thereby promoting safer and purposeful motion planning. In the implementation, techniques like utilizing inverse kinematics solver for path guidance and sparse reward to expedite reaching time are integrated into the MPPI cost function design. Comparative evaluations against the traditional MPPI architecture and standard SDF cost designs demonstrate the superiority of the new method. Real-world experiments, including human–robot interaction, obstacle-crossing, and grasping tasks, validate the robustness and universality of our methodology, with average and maximum end effector speeds of 0.523 m/s and 1.225 m/s respectively.**

***Index Terms*—Dynamic obstacle avoidance, model predictive path integral (MPPI), reactive motion planning, signed distance field (SDF), redundant manipulators.**

I. Introduction

THE challenges associated with dynamic obstacle avoidance for robotic manipulators present a complex and pivotal issue, with real-time motion planning algorithm and environment perception methods emerging as prominent considerations [1], [2], [3]. In the domain of real-time planning, conventional global path planning methods, such as rapidly-exploring random trees (RRT) and probabilistic roadmaps [4], aim to find complete paths from the robot's initial position to its destination by considering the entire environmental space. However, in the high-dimensional space of robotic manipulators, these approaches may encounter computational challenges, impeding real-time planning feasibility. Reactive motion planning algorithms, particularly those employing model-predictive control (MPC) [5], [6], [7], provide a practical alternative by swiftly generating executable local paths within a relatively short timeframe. This showcases enhanced adaptability to the rapidly changing environment. Model predictive path integral (MPPI) [8], [9], [10], a variant of MPC, was proposed to sample control sequences and continuously update the control policy based on sample weights. This approach circumvents challenges associated with computing the gradient of the control objective function, demonstrating robust real-time performance, especially when leveraging GPU hardware support.

However, MPPI's reliance on a single sequence of Gaussian means to sample trajectories constrains its exploration capability, potentially leading the robot to be trapped in local minima and compromising the safety of dynamic obstacle avoidance [11], [12], [13]. To avoid this issue, some approaches attempt to reshape the MPPI framework and integrate other algorithmic tools, such as SDF gradient [3], conditional value at risk (CvaR) [14], and control barrier functions (CBF) [15], to further modify the path planned by the MPPI, which are more similar to sequential optimization methods.

In the context of collision detection for robotic manipulators, two main approaches are used. One is the mesh-based collision detection methods [16], [17], which feature with high precision but a significant computational cost, limiting their real-time applicability. The other approach, point clouds based collision detection (e.g., Voxel Grid, Octree [18]), divides 3-D space into regular voxel grids, enabling faster detection but sacrificing accuracy. While being suitable for real-time collision detection, these voxel-based methods [19], [20], [21] only offer binary feedback on collisions, lacking feedforward information like

Received 1 March 2025; accepted 20 July 2025. Date of publication 19 August 2025; date of current version 8 September 2025. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB4704600, in part by the Shandong Provincial Natural Science Foundation for Distinguished Young Scholars under Grant ZR2022JQ28, and in part by the National Natural Science Foundation of China under Grant 62373221. This article was recommended for publication by Associate Editor W. Wan and Editor T. Bretl upon evaluation of the reviewers' comments. *(Corresponding author: Lelai Zhou.)*

Lelai Zhou, Zhengmao Li, and Yibin Li are with the Center for Robotics, School of Control Science and Engineering, Shandong University, Jinan 250061, China, and also with the Engineering Research Center of Intelligent Unmanned System, Ministry of Education, Jinan 250061, China (e-mail: zhoulelai@sdu.edu.cn; zeman@mail.sdu.edu.cn; liyb@sdu.edu.cn).

Shaoping Bai is with the Department of Materials and Production, Aalborg University, 9220 Aalborg, Denmark (e-mail: shb@mp.aau.dk).

This article has supplementary downloadable material available at https://doi.org/10.1109/TRO.2025.3600125, provided by the authors.

Digital Object Identifier 10.1109/TRO.2025.3600125

1941-0468 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

collision risk or obstacle distance. Despite current advanced approaches attempt to leverage neural networks for faster collision detection [2], [22], [23], these approaches still suffer from low information density due to their voxel-based nature that can only provide Boolean value.

SDF has potential to overcome this challenge. SDF determines the closest distance from each voxel grid point to obstacles. The representation of SDF maps can take an explicit form using three-dimensional data [24], [25] or an implicit form using neural networks [3], [26], [27]. The advancements in GPU and other hardware capabilities in recent years have alleviated the computational cost of SDF calculations, making SDF widely applied in the collision checking area. However, its application in robotic manipulator obstacle avoidance, especially when integrated with the MPPI, was rarely explored.

This article presents a parallel MPPI (PMPPI) motion planning algorithm integrated with SDF for dynamic obstacle avoidance tasks. Unlike previous sequential optimization methods, the PMPPI algorithm concurrently runs multiple MPPI planners with different strategies. This allows for the adaptive integration of planned paths generated by different strategies based on the current state, enhancing the exploration capability of the MPPI. Furthermore, several cost functions are tailored for the application of the PMPPI in dynamic obstacle avoidance tasks. First, by integrating SDF into the MPPI cost function, accounting for both robot velocity and SDF gradient, we enhance the MPPI's environmental awareness. This enables more precise obstacle avoidance maneuvers. In addition, we present a real-time multiprocessing approach leveraging the inverse kinematics (IK) solver, namely TRAC-IK [28] to assist the robotic manipulator in navigating away from local minima while optimizing its operability. Moreover, a reinforcement learning technique called sparse reward [29] is integrated into the cost function to minimize the end-pose error. Our objective is to enable a 7-DOF robotic manipulator to navigate to a target pose while avoiding sudden or dynamically moving obstacles in its workspace, as illustrated in Fig. 1.

The main contributions of our work are as follows.

1) *Proposing a fast and safe path planning algorithm PMPPI*: By introducing the value function of the MPPI and Judge strategy concepts into the multimodal framework, the PMPPI can leverage the advantages of different strategies and demonstrates superior exploration capabilities compared to the traditional MPPI methods.
2) *Defining a new cost function: gradient-velocity modulated SDF cost (GVM-SDF)*: Previous approaches incorporating SDF into the MPPI cost function often led to issues where the robot would either rapidly pass through obstacles or get stuck close to them. Our method introduces dynamic modulation of the SDF cost based on both robot velocity and SDF gradient, enabling safer and more responsive motion performance.
3) *Experimental Validation*: We conducted real-world experiments to validate our approach's exceptional dynamic obstacle avoidance capabilities, including human–robot interaction, obstacle-crossing, and grasping tasks with the physical Franka robot. Results showed average and maximum end effector (EE) speeds of 0.523 m/s and 1.225 m/s, respectively, at a control frequency of 27.8 Hz, with Crash rate below 5% (see Fig. 1).

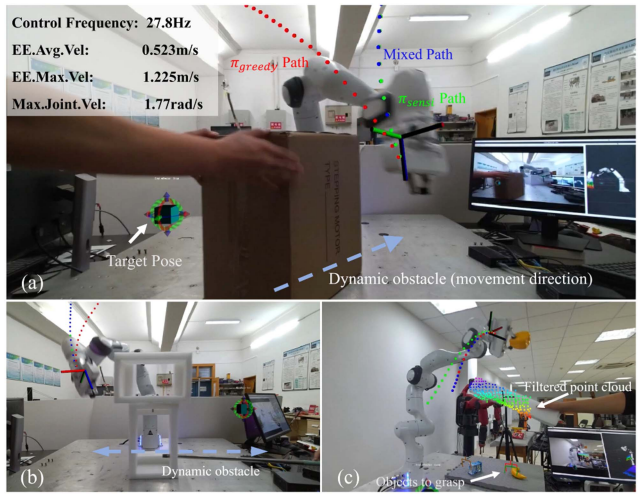


Fig. 1. Scenarios of dynamic obstacle avoidance using the PMPPI framework. (a) Obstacle suddenly appears in the robot's path, testing real-time response to unexpected events. (b) Dynamic obstacle crosses the workspace, requiring immediate trajectory adjustment. (c) Obstacle interferes with a pick-and-place task, evaluating the system's ability to balance task completion and dynamic avoidance.

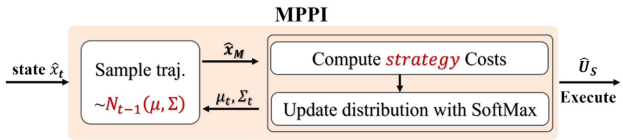


Fig. 2. MPPI architecture.

The rest of this article is organized as follows. Some fundamental works are outlined in Section II. Section III provides a detailed problem statement, followed by the introduction of the PMPPI framework in Section IV. Cost functions for the PMPPI application in dynamic obstacle tasks are defined in Section V. In Section VI, the evaluation of the new approach through obstacle avoidance tests on both 2-D PointMass and 7-D Franka platforms is described. Section VII presents our real-world experiments. The research work is discussed in Section VIII. Finally, Section IX concludes this article.

II. Related Work

A. MPPI Framework

The MPPI framework is structured with three pivotal components: trajectory generation, cost function computation, and distribution update with softmax weight, as illustrated in Fig. 2.

1) Trajectory Generation: Trajectory generation starts with sampling the control sequences $\{u_{i,h}\}_{i=1,h=1}^{M,H}$ from the policy distribution at the previous time step $t-1$, represented by a sequence of Gaussian distributions $\mathcal{N}(\hat{\boldsymbol{\mu}}_{t-1}, \hat{\boldsymbol{\Sigma}}_{t-1})$ over a look-ahead horizon H. Here, the index i corresponds to M different trajectories, while h denotes the time step within each trajectory

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

over the horizon. The control sequences are sampled for each time step h across the total horizon H, and for each trajectory i, from a distribution parameterized by the mean and covariance of the previous time step. The control sequences are given by

$$u_{i,h} \sim \mathcal{N}(\hat{\mu}_{t-1,h}, \hat{\Sigma}_{t-1,h}) \quad \forall i \in [1, M], h \in [1, H] \tag{1}$$

where $\hat{\boldsymbol{\mu}}_{t-1} = (\hat{\mu}_{t-1,1}, \hat{\mu}_{t-1,2}, \ldots, \hat{\mu}_{t-1,H})$ and $\hat{\boldsymbol{\Sigma}}_{t-1} = (\hat{\Sigma}_{t-1,1}, \hat{\Sigma}_{t-1,2}, \ldots, \hat{\Sigma}_{t-1,H})$ represent the mean and covariance sequences at the previous time step. These sequences are used to roll out M trajectories by predicting future state sequences $\{\hat{x}_{i,h}\}_{i=1\ldots M}^{h=1\ldots H}$ using the system's dynamic model, starting from the current state $\hat{x}_t$. Each trajectory is denoted as $\hat{\boldsymbol{x}}_i = (\hat{x}_{i,1}, \hat{x}_{i,2}, \ldots, \hat{x}_{i,H})$, where $\hat{x}_{i,1} = \hat{x}_t$. The robot state $\hat{x}_{i,h}$ includes joint position, velocity, and acceleration $[q_{i,h}, \dot{q}_{i,h}, \ddot{q}_{i,h}]$, the EE Cartesian pose $X_{i,h}^{\text{ee}} \in \text{SE}(3)$, and other relevant information (as discussed in [9]).

2) Cost Function Computation Based on Strategy: After generating trajectories, the MPPI computes the cost of each trajectory based on its planning strategy (preference), such as prioritizing rapid target reaching or enhanced obstacle avoidance. For each sampled trajectory $\{\hat{x}_{i,h}\}_{i=1\ldots M}^{h=1\ldots H}$, predefined cost functions are employed to evaluate performance. The total cost for the ith trajectory, denoted $\text{Cost}(\hat{\boldsymbol{x}}_i)$, is the summation of costs at each state

$$\text{Cost}(\hat{\boldsymbol{x}}_i) = \sum_{h=1}^{H} \text{cost}(u_{i,h}, \hat{x}_{i,h}) \quad \forall i \in [1, M]. \tag{2}$$

3) Distribution Update With Softmax Weight: The policy distribution is updated using the softmax function applied to the trajectory costs. The weight for each trajectory is

$$w_i = \frac{\exp\left(-\frac{1}{\lambda} \cdot \text{Cost}(\hat{\boldsymbol{x}}_i)\right)}{\sum_{j=1}^{M} \exp\left(-\frac{1}{\lambda} \cdot \text{Cost}(\hat{\boldsymbol{x}}_j)\right)} \quad \forall i \in [1, M]. \tag{3}$$

The updated policy distribution parameters are then computed

$$\begin{aligned} \hat{\mu}_{t,h} &= (1 - \alpha_\mu)\hat{\mu}_{t-1,h} + \alpha_\mu \sum_{i=1}^{M} w_i u_{i,h} \\ \hat{\Sigma}_{t,h} &= (1 - \alpha_\sigma)\hat{\Sigma}_{t-1,h} + \alpha_\sigma \sum_{i=1}^{M} w_i \delta_{i,h} (\delta_{i,h})^T \end{aligned} \tag{4}$$

where $\delta_{i,h} = u_{i,h} - \hat{\mu}_{t,h}$. λ is the temperature parameter that controls the exploration–exploitation tradeoff, and α_μ and α_σ are the learning rates for the mean and covariance, respectively. The Gaussian mean at the first time step $(\widehat{U}_s = \widehat{\mu}_{t,1})$ is subsequently deployed for execution.

B. Adjust the MPPI Framework

To address the insufficient exploration of the MPPI, several approaches have been proposed. These include methods such as using CBF [15] or SDF Gradient [3] to steer the MPPI planned control sequence away from obstacles, utilizing CVaR to sample around and assess the collision risk of the MPPI sampled trajectories [14], and redesigning the covariance of the control random distribution to enhance the MPPI's exploration capability [30]. These methods can be seen as forms of sequential optimization methods: Instead of directly using the MPPI planned path as a control sequence, they reshape the MPPI framework and integrate other optimization algorithms to further modify it.

It is logically conceivable that we can also employ the MPPI as a subsequent optimization module. This architecture connects two tiers of the MPPI in series, with control sequences from the first tier serving as the sampling mean for the second. In this setup, the original MPPI with a greedy planning strategy (π_{greedy}) aims to reach the target quickly but may risk collisions. By substituting the greedy strategy with one more sensitive to the environment (π_{sensi}), the path planned by π_{greedy} MPPI can be pushed away from obstacles by π_{sensi} MPPI. While this setup theoretically enhances exploration, it also introduces additional computational costs due to the added steps involving sampling and rolling out the dynamic model at each optimizer iteration. Thus, there is a necessity to devise a more efficient algorithm architecture capable of concurrently running multiple MPPI planners with only one sampling process per iteration.

In the work [31], a multimodal approach was introduced to differential dynamic programming (DDP), utilizing softmax functions to allocate blending weights to each instance of DDP based on their respective value functions. These value functions assess the quality or performance of paths generated by each instance of DDP with varying strategies. In addition, the work [11] derived the value function of the MPPI to quantify the quality of its planned trajectories in the current state. Building on the methodologies proposed in [31] and [11], we extend these concepts to develop a multimodal framework for MPPI. This framework integrates the advantages of diverse control strategies and demonstrates superior exploration capabilities compared to traditional MPPI methods.

C. SDF Cost Function Design

In [9], a specific implementation method of MPPI, called STORM, was developed for robotic manipulors. The STORM was further included in NVIDIA's lib called SceneCollisionNet [23] to design the environment collision cost. However, SceneCollisionNet only provides binary value, limiting its ability to foresee dynamic environment changes. To enhance MPPI environment awareness capability, several methods have been proposed to integrate SDF into the MPPI cost function for robotic manipulators, yet they still encounter various challenges. For instance, NJSDF [26] utilizes neural networks to construct SDF maps, but its SDF cost function design reverts to a discrete (binarized) approach, penalizing only whether a collision occurs or not. The SDF cost function used in RAMP [3] simply aggregates SDF Potential over time, which may prompt the robot to traverse high-cost areas rapidly as a planning preference to lower the overall cost. Even though RAMP [3] also utilizes SDF gradient information to modify the path generated by the MPPI, the implementation is essentially as a sequential addition of SDF cost rather than an effective integration with the MPPI. Similarly, in the domain of sample-based algorithms, the SDF cost function implemented in STOMP [24], which multiplies the SDF Potential by the robot's velocity, can potentially lead the robot to reduce its speed to zero near obstacle regions in an

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

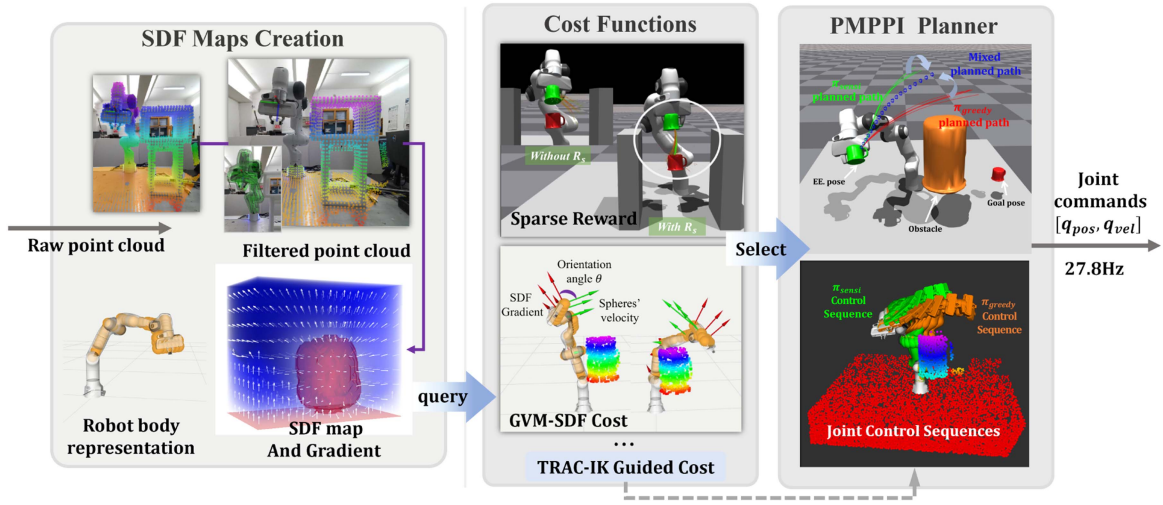


Fig. 3. Architecture overview of the PMPPI.

attempt to minimize the cost. In addition, CHOMP [25] incorporates SDF potential fields and gradient information into the design of collision cost functions. However, the derivation of its formulas appears redundant, lacks clarity, and poses challenges for comprehension.

III. Problem Statement

With the objective to enable a 7-DOF robotic manipulator to navigate to a target pose while avoiding sudden or dynamically moving obstacles in its workspace (see Fig. 1), the motion planning method will be implemented for a purely vision-based perception system with depth cameras, without relying on any preknown environmental collision models or QR codes for obstacle localization, thereby refining its real-time responsiveness and adaptability in dynamic environments. The inputs consist of workspace point clouds and the current joint states $[q_t, \dot{q}_t]$, while the outputs are joint commands $[q_t^d, \dot{q}_t^d]$. The PD-controlled manipulator translates these joint commands into joint torques

$$\tau_t = C(q_t)\dot{q}_t + K_p(q_t^d - q_t) + K_d(\dot{q}_t^d - \dot{q}_t) \tag{5}$$

where τ_t is the torque at time t, $C(q_t)$ denotes the Coriolis force matrix, K_p and K_d are used to minimize the error between the desired joint states q_t^d and actual joint states q_t.

To achieve this goal, we divide our approach into two subgoals. One is to reshape the MPPI framework to enhance its exploration capability. The proposed PMPPI needs to address the challenge of appropriately mixing planned paths $U_s^{(i)}$ generated by N MPPI planners, which can be represented as

$$U^* = \sum_{i=1}^{N} w_i \cdot U_s^{(i)} \tag{6}$$

where U^* is the optimal control sequence, $U_s^{(i)}$ is the control sequence from the nth MPPI planner, w_i are the weights assigned to each planner's output, and N is the number of planners. The other is to redesign the SDF cost function to enhance the MPPI environment awareness capability. The proposed GVM-SDF needs to find the relationship between the SDF gradient and the robot's velocity during the running process. This will make the cost function more risk-aware and help the system escape local minima. This concept can be formulated as

$$C_{\text{env}}(x, \dot{x}, \text{SDF}) = g(\nabla \text{SDF}(x), \dot{x}) \tag{7}$$

where x is the robot position, $\dot{x}$ is the velocity, and $\nabla \text{SDF}(x)$ represents the gradient of the SDF at robot's current position.

In Fig. 3, we delineates our motion planning architecture into three primary components: the perception layer, the planning layer, and the execution layer.

The perception layer begins by collecting point clouds from camera and subsequently filters out point clouds relevant to the robotic manipulator itself and those outside working space. This is followed by voxelization and the sequential computation of the SDF, potential, and gradient maps. Robots are commonly represented by a set of overlapping spheres [3], [24], [26], corresponding to this collision detection method.

The planning layer utilizes the designed PMPPI algorithm, which blends paths planned by the greedy planning strategy π_{greedy} and sensitive strategy π_{sensi} planners. This algorithm dynamically adjusts the blending weights according to the current environmental state, resulting in a mixed planned path. The cost function for each MPPI planner is formed by selecting or weighting together multiple subcost functions, which include the GVM-SDF environment collision cost (involving the calculation of cost between key points of the robotic manipulator and SDF/gradient map), the TRAC-IK guided cost, the sparse reward R_s, dynamic constraint cost, and others. Different strategies express their uniqueness through the tailored design of their respective cost functions.

The PMPPI algorithm samples control commands in the acceleration space $(\ddot{q}_t^d)$ for enhanced motion smoothness. These commands are then integrated to produce the desired joint positions and velocities $[q_t^d, \dot{q}_t^d]$, which are passed to the execution layer. The execution layer converts these desired joint states into joint torques for actuation using a PD-based control law.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

IV. Parallel MPPI

In this section, a comprehensive mathematical formulation of the PMPPI algorithm is presented. Detailed theoretical foundations, including the multimodal extensions from MME-DDP [31], MPQ [11], and the transition to Gaussian mixture regression (GMR) [32], are included in Appendix A.

A. Mathematical Model of the PMPPI

Standard MPPI is able to perform real-time trajectory optimization through stochastic sampling, but its single-planner approach may overlook diverse solutions in multimodal environments. Drawing from MME-DDP's multimodal weighting and GMR's distribution approximation, we propose PMPPI. By integrating multiple(N) MPPI planners, each of which employs a unique strategy (e.g., π_{greedy} for rapid goal-seeking, π_{sensi} for risk aversion), PMPPI can produce a policy that adapts to the current state $\hat{x}_t$ while ensuring fairness across strategies.

1) Multimodal Policy Representation: The PMPPI policy begins as a Gaussian mixture, reflecting the diversity of planner outputs

$$\hat{\pi}(u|\hat{x}_t) = \sum_{n=1}^{N} w^{(n)} \hat{\pi}^{(n)}(u|\hat{x}_t) \tag{8}$$

where $\hat{\pi}^{(n)}(u|\hat{x}_t) \sim \mathcal{N}(\hat{\mu}_t^{(n)}, \hat{\Sigma}_t^{(n)})$ is the policy of the nth planner, with mean $\hat{\mu}_t^{(n)} = [\hat{\mu}_{t,1}^{(n)}, \ldots, \hat{\mu}_{t,H}^{(n)}]^\top$ and covariance $\hat{\Sigma}_t^{(n)} = \text{diag}(\hat{\Sigma}_{t,1}^{(n)}, \ldots, \hat{\Sigma}_{t,H}^{(n)})$ over a planning horizon H, indexed by $n \in [1, N]$. $w^{(n)}(t, \hat{x}_t)$ is the dynamic weight, akin to MME-DDP's, but tailored to MPPI's sampling context.

This mixture captures multiple control possibilities, such as direct paths versus detours around obstacles, allowing PMPPI to explore a broader solution space than standard MPPI.

2) Policy Approximation and Updates: Given the computational demands of real-time control, we approximate $\hat{\pi}(u|\hat{x}_t)$ as a single Gaussian, inspired by GMR's practical simplification

$$\hat{\pi}(u|\hat{x}_t) \approx \mathcal{N}(\hat{\mu}_t, \hat{\Sigma}_t) \tag{9}$$

where $\hat{\mu}_t = [\hat{\mu}_{t,1}, \ldots, \hat{\mu}_{t,H}]^\top$ and $\hat{\Sigma}_t = \text{diag}(\hat{\Sigma}_{t,1}, \ldots, \hat{\Sigma}_{t,H})$ represent the policy parameters over H timesteps. They are updated iteratively to adapt to changing conditions

$$\begin{aligned}
\hat{\mu}_{t,h} &= (1-\alpha_\mu)\hat{\mu}_{t-1,h} + \alpha_\mu \sum_{n=1}^{N} w^{(n)} \hat{\mu}_{t,h}^{(n)} \\
\hat{\Sigma}_{t,h} &= (1-\alpha_\sigma)\hat{\Sigma}_{t-1,h} - \alpha_\sigma \hat{\mu}_{t,h}\hat{\mu}_{t,h}^\top \\
&\quad + \alpha_\sigma \sum_{n=1}^{N} w^{(n)} \left(\hat{\Sigma}_{t,h}^{(n)} + \hat{\mu}_{t,h}^{(n)} \hat{\mu}_{t,h}^{(n)\top}\right)
\end{aligned} \tag{10}$$

for $h = 1, \ldots, H$. Here, $\alpha_\mu, \alpha_\sigma \in (0, 1)$ are smoothing factors that balance historical consistency with new planner inputs, ensuring stability across iterations.

This approximation, while reducing multimodal richness, mirrors GMR's strategy of collapsing a mixture into a unimodal distribution for efficiency. Unlike GMR, which derives components from a statistical joint model, PMPPI's $\hat{\pi}^{(n)}(u|\hat{x}_t)$ emerges from MPPI's sampling and optimization, blending statistical inspiration with control practicality.

3) Weight Computation With Judge Strategy: The weights $w^{(n)}(t, \hat{x}_t)$ are computed using a softmax function, extending MPQ's value-based approach [11]

$$w^{(n)}(t, \hat{x}_t) = \frac{\exp\left(-\frac{1}{\alpha} V^{(n)}(t, \hat{x}_t)\right)}{\sum_{j=1}^{N} \exp\left(-\frac{1}{\alpha} V^{(j)}(t, \hat{x}_t)\right)} \quad \forall n \in [1, N] \tag{11}$$

where $\alpha > 0$ is a temperature parameter adjusting the sharpness of differentiation. Lower values favor top performers, while higher values distribute weights more evenly. $V^{(n)}(t, \hat{x}_t)$ is the nth MPPI planner's value function at current state $\hat{x}_t$, used to quantify the nth planner's performance, which can be formulated as follows:

$$\begin{aligned}
V^{(n)} = -\lambda \log E\Bigg[\exp\Bigg(-\frac{1}{\lambda}\Bigg(\sum_{h=0}^{H-1} c_{\pi^*}(u_{i,h}, \hat{x}_{i,h}) \\
+ Q^{\pi^*}(u_{i,h}, \hat{x}_{i,h}) \Bigg)\Bigg)\Bigg]. \quad i \in [1, M].
\end{aligned} \tag{12}$$

A key feature in PMPPI is Judge strategy π^*, designed to evaluate trajectories consistently across all planners, mitigating biases inherent in planner-specific metrics. Here, unlike the value function derived in MPQ [11], which directly computes the cost value using the nth MPPI's internal standard. The quantification (scoring) and comparison (assessment) of trajectories planned by various strategies (e.g., $\pi_{\text{greedy}}, \pi_{\text{sensi}}$) should be conducted under the same criteria. Therefore, we design the Judge strategy π^* as an independent and objective evaluation standard to ensure fairness in the evaluation and comparison of different strategies.

We define the cost of the kth trajectory evaluated on the Judge strategy π^* as

$$\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_k) = \sum_{h=1}^{H} c_{\pi^*}(u_{i,h}, \hat{x}_{i,h}) + Q^{\pi^*}(u_{i,h}, \hat{x}_{i,h}) \tag{13}$$

where $Q^{\pi^*}(u_{i,h}, \hat{x}_{i,h})$ is the terminal function. Then, the value function can be expressed as

$$\begin{aligned}
V^{(n)} &= -\lambda \log E\left[\exp\left(-\frac{1}{\lambda}\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_k)\right)\right] \\
&\approx -\lambda \log\left(\frac{1}{K}\sum_{k=1}^{K} \exp\left(-\frac{1}{\lambda}\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_k)\right)\right)
\end{aligned} \tag{14}$$

these K trajectories are generated by the nth MPPI planner, which ranks its trajectories' cost based on its own planning strategy and selects the trajectories with the lowest costs (i.e., the top-K trajectories) for the evaluation process. λ influences the sensitivity of the value function to changes in the cost, playing a role in the exploration–exploitation balance.

This judge strategy ensures that $V^{(n)}(t, \hat{x}_t)$ reflects an objective standard, aligning with MME-DDP's performance-driven weighting while enhancing equity across diverse strategies.

However, there exists one problem. When the trajectory distributions of the top-K trajectories generated by different strategies overlap significantly, assigning a uniform weight $\frac{1}{K}$ to each

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

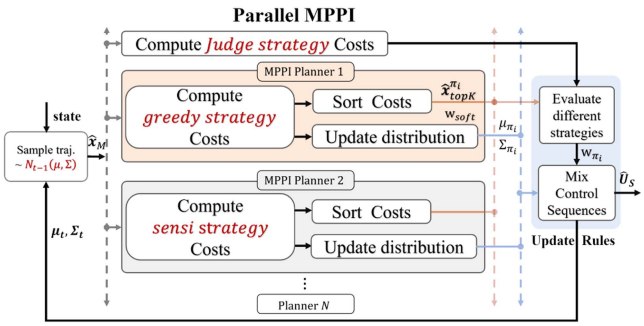


Fig. 4. PMPPI Architecture simplifies optimization by concurrently running multiple MPPI planners with different strategies, requiring only one sampling step per iteration.

trajectory may lead to identical quantifications, thus failing to effectively capture the differences between each MPPI planner. Therefore, we introduce a strategy-oriented softmax weight, denoted as $w_{\text{soft}}(\hat{\boldsymbol{x}}_k)$, for each trajectory

$$w_{\text{soft}}(\hat{\boldsymbol{x}}_k) = \frac{\exp\left(-\frac{1}{\beta}\text{Cost}^{(n)}(\hat{\boldsymbol{x}}_k)\right)}{\sum_{l=1}^{K}\exp\left(-\frac{1}{\beta}\text{Cost}^{(n)}(\hat{\boldsymbol{x}}_l)\right)} \qquad \forall k \in [1, K]. \tag{15}$$

Here, $\text{Cost}^{(n)}(\hat{\boldsymbol{x}}_k)$ is the cost of the kth trajectory determined by the nth MPPI planner's own strategy. This cost is internal to the planner and represents its native evaluation of the trajectory's quality, without the influence of the Judge strategy. β controls the distribution of weights assigned to the topK trajectories, allowing for a nuanced extraction of planners' strategy differences before evaluation by the Judge strategy.

In short, when submitting trajectories to the Judge strategy for evaluation, each MPPI planner provides not only the planned topK trajectories but also the corresponding *softmax weight*, as depicted in Fig. 4, which showcases the data flow of the PMPPI. Considering potential numerical overflow, the value function $V^{(n)}$ for the nth MPPI planner, which incorporates these weights, can be derived as

$$\begin{aligned} V^{(n)} &= -\lambda \log\left(\sum_{k=1}^{K} w_{\text{soft}}(\hat{\boldsymbol{x}}_k)\exp\left(-\frac{1}{\lambda}\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_k)\right)\right) \\ &= -\lambda \log\sum_{k=1}^{K}\exp\left(\boldsymbol{A}_k - A_{\max}\right) - \lambda A_{\max} \\ &\quad +\lambda \log\sum_{k=1}^{K}\exp\left(\boldsymbol{B}_k - B_{\max}\right) + \lambda B_{\max} \end{aligned} \tag{16}$$

where

$$\boldsymbol{A}_k = -\frac{1}{\beta}\text{Cost}^{(n)}(\hat{\boldsymbol{x}}_k) - \frac{1}{\lambda}\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_k)$$

$$\boldsymbol{B}_k = -\frac{1}{\beta}\text{Cost}^{(n)}(\hat{\boldsymbol{x}}_k).$$

Here, $A_{\max}$ and $B_{\max}$ represent the maximum values of all $\boldsymbol{A}_k$ and $\boldsymbol{B}_k$, respectively.

4) Action Selection: The final control action is extracted as

$$u_t = \hat{\mu}_{t,1}$$

which is the mean of the approximated policy at the first timestep. This choice leverages the multimodal contributions of all planners, distilled into a single, executable command, consistent with real-time robotic requirements.

Remark 1: The parameters α, λ, and β play distinct roles in enabling the flexibility and adaptability of the PMPPI.

1) α significantly influences the differentiation between planners based on their value functions.
2) λ directly affects the sensitivity of the value function to cost variations.
3) β allows for a more detailed representation of strategy differences, by fine-tuning the weight distribution for the top-K trajectories.

Remark 2: The Judge strategy π^*, distinct from each MPPI planner's own planning strategy (e.g., π_{greedy}, π_{sensi}), does not participate in generating planning path step, solely evaluating submitted trajectories based on their distance to target pose and risk of collisions with environment in dynamic obstacle task.

B. Implementation of the PMPPI

The PMPPI has been implemented, as presented in Algorithm 1, following the architecture in Fig. 4. The PMPPI framework is composed of three core components: cost function computation for each strategy, evaluation of different strategies, and update rules.

The PMPPI algorithm begins by sampling control sequences (Lines 2–6) from multiple Gaussian distributions, including those maintained by individual MPPI planners and a globally mixed distribution. Specifically, for each MPPI planner n, a set of control sequences $\{u_{i,h}^{(n)}\}_{i=1}^{M}$ is sampled from the planner's Gaussian distribution $\mathcal{N}(\widehat{\mu}_{t-1,h}^{(n)}, \widehat{\Sigma}_{t-1,h}^{(n)})$. In addition, a globally mixed Gaussian distribution $\mathcal{N}(\widehat{\mu}_{t-1,h}, \widehat{\Sigma}_{t-1,h})$ is used to sample controls $\widehat{u}_{i,h}$. These sampled controls are then propagated through the dynamic model to generate state trajectories $\hat{\boldsymbol{x}}_M$ of length H [see Fig. 5(a) for sampled trajectories].

Next, the algorithm computes costs for each strategy (Lines 7–14). For each MPPI planner and the Judge strategy, S predefined subcost functions are evaluated in parallel

$$\begin{aligned} \text{SubCost}^{(s)}(\hat{\boldsymbol{x}}_M) &= \sum_{h=1}^{H}\text{subcost}^{(s)}(u_{j,h}, \hat{x}_{j,h}) \\ & j = 1, \ldots, M. \end{aligned} \tag{17}$$

These subcosts are then weighted and aggregated to form the total cost for each strategy

$$\begin{aligned} \text{Cost}^{\pi}(\hat{\boldsymbol{x}}_M) &= \sum_{s=1}^{S} w_{\pi}^{(s)}\text{SubCost}^{(s)}(\hat{\boldsymbol{x}}_M) \\ & \pi \in [\pi^*, \{\pi^{(n)}\}_{n=1}^{N}]. \end{aligned} \tag{18}$$

This step ensures that each strategy's performance is quantified based on its specific cost function.

The algorithm then evaluates different strategies (Lines 15–22). For each MPPI planner n with strategy $\pi^{(n)}$, the basic MPPI algorithm updates the policy distribution parameters $\widehat{\boldsymbol{\mu}}_{\pi_n,t}$ and

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

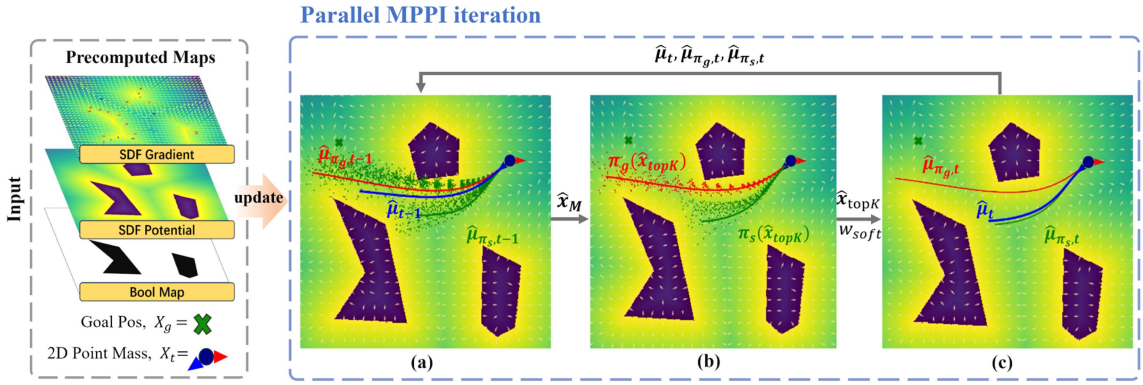


Fig. 5. Illustration of one PMPPI iteration using a 2-D PointMass with red (gradient) and blue (velocity) arrows. Background: SDF gradient and potential maps overlaid, updated in real time. Scattered points: Trajectories from the PMPPI sampling. (a) Sample trajs. $\hat{x}_M$. (b) Evaluate different policies. (c) Mixed $\hat{\mu}_t$ update.

$\widehat{\Sigma}_{\pi_n,t}$ using the softmax function applied to trajectory costs $\text{Cost}^{\pi^{(n)}}(\hat{\boldsymbol{x}}_M)$. The planner selects the top-K trajectories $\hat{\boldsymbol{x}}_{\text{topK}}$ with the lowest costs and assigns them softmax weight $w_{\text{soft}}(\hat{\boldsymbol{x}}_k)$. The Judge strategy evaluates these selected trajectories using its own cost function (16). This step ensures that the most promising trajectories are identified and weighted appropriately [visualized in Fig. 5(b)].

Finally, the update rules are applied (Lines 23-End). Based on the value function $V^{\pi^{(n)}}$, the algorithm computes the final mixed weights for each planner and updates the mixed Gaussian means $\widehat{\boldsymbol{\mu}}_t$ and covariance $\widehat{\boldsymbol{\Sigma}}_t$ [see (10)].

Fig. 5 illustrates one PMPPI iteration using a 2-D PointMass, demonstrating how PMPPI balances exploration and optimization across strategies. In Fig. 5(a), sampled trajectories $\hat{\boldsymbol{x}}_M$ are shown as green scatter points, with the evolution of mixed Gaussian means ($\hat{\mu}_t$, $\hat{\mu}_{\pi_g,t}$, $\hat{\mu}_{\pi_s,t}$) depicted as blue, red, and green lines, respectively, highlighting the diversity of sampled paths from multiple distributions. Fig. 5(b) displays the top-K trajectories ($\hat{x}_{\text{topK}}$) with lowest costs, marked as red points for the greedy strategy(π_g) and green points for the sensitive strategy(π_s), reflecting the Judge strategy's evaluation of planner performance.

The first time-step of the updated Gaussian mean $\widehat{\mu}_{t,h=1}$ is sent to the actuators, and the policy distribution is rolled forward for the next iteration.

Remark 3: GMR is adopted to determine the control output u given the current state $\hat{x}_t$. During the PMPPI sampling process, trajectory sampling consists of two complementary components.

1) *Intramodal Sampling:* Each planner independently generates samples from its own Gaussian distribution

$$\hat{u}_{i,h}^{(n)} \sim \mathcal{N}(\hat{\mu}_{t-1,h}^{(n)}, \hat{\Sigma}_{t-1,h}^{(n)}) \tag{19}$$

where $\hat{\mu}_{t-1,h}^{(n)}$ and $\hat{\Sigma}_{t-1,h}^{(n)}$ denote the mean and covariance of the planner n at time step $t-1$.

2) *Mixed Distribution Sampling:* A globally mixed Gaussian distribution is constructed using GMM-GMR, from which samples are drawn as follows:

$$\hat{u}_{i,h} \sim \mathcal{N}(\hat{\mu}_{t-1,h}, \hat{\Sigma}_{t-1,h}). \tag{20}$$

These two sampling mechanisms, respectively, preserve the unique modal characteristics of each planner (via intramodal sampling) and integrate global multimodal features (via mixed distribution sampling), ensuring both diversity and adaptability in trajectory generation.

V. Cost Function Design in the MPPI

In the PMPPI, multiple subcost functions have to be selected or weighted together to form the cost function for each MPPI planner based on their respective strategies. In particular, the GVM-SDF Cost aims to improve the MPPI's risk-awareness capability, while the TRAC-IK guided cost helps avoid Cartesian local minima during the running process. In addition, sparse reward is adopted to minimize the EE pose errors.

A. GVM-SDF Collision Cost

The GVM-SDF cost function is developed based on the SDF maps (SDF map, SDF-Potential map, SDF-Gradient map), as well as the representation of the robot body and its velocity.

1) Precomputed SDF Maps: The SDF map represents, for each voxel point x, the minimum distance to the nearest obstacle boundary. The SDF map $d(x)$ obtained by applying a signed Euclidean distance transform to a voxel grid, obtained directly from a point clouds. Inspired by the application of the inflation layer in car navigation [33], we adapt a similar concept to robotic manipulators. Building upon the SDF Map $d(x)$, we design a corresponding workspace potential function, also referred to as the SDF-Potential map $\Phi(x)$, for a finer control of the distance between the robot and obstacles, providing increased safety margins near obstacles

$$\Phi(x) = \begin{cases} 1 & \text{if } d(x) < \sigma_1 \\ \exp(-\kappa(d(x) - \sigma_1)) & \text{if } \sigma_1 \le d(x) \le \sigma_2 \\ 0 & \text{otherwise} \end{cases} \tag{21}$$

where κ adjusts the descending speed, σ_1 and σ_2 correspond to the inscribed radius and inflation radius within the inflation layer, respectively. The SDF-Gradient map $\text{Grad}(x)$ indicates the direction of steepest change at each point in the SDF map $d(x)$. We compute the gradient in the $r \in (\vec{x}, \vec{y}, \vec{z})$ direction

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Parallel Model Predictive Path Integral Control (PMPPI).

Declare:
$\pi^{(n)}$: the n-th MPPI planner's strategy, $n \in [1, N]$;
π^*: Judge strategy;
β, λ, α : Temperature parameters;
$topK$: Num. of evaluation trajs. each MPPI submit;
S: Num. of subcost functions;
$\boldsymbol{u}_M, \hat{\boldsymbol{x}}_M$: Sampled M action and state trajs. of len. H;
$\widehat{\boldsymbol{\mu}}_t, \widehat{\boldsymbol{\Sigma}}_t$: Sequence of final mixed Gauss. means, Covs.;
$\widehat{\boldsymbol{\mu}}_{\pi^{(n)},t}, \widehat{\boldsymbol{\Sigma}}_{\pi^{(n)},t}$: Gauss. means, Covs. w.r.t. $\pi^{(n)}$;
while *task not completed* **do**
  ▷ **Sample control sequences:**
  for each MPPI planner $n = 1, \ldots, N$ **do**
    $\hat{u}^{(n)}_{i,h} \sim \mathcal{N}(\widehat{\mu}^{(n)}_{t-1,h}, \widehat{\Sigma}^{(n)}_{t-1,h})$
  end for
  $\widehat{u}_{i,h} \sim \mathcal{N}(\widehat{\mu}_{t-1,h}, \widehat{\Sigma}_{t-1,h})$
  ▷ **Generate state trajectories:**
  $\hat{\boldsymbol{x}}_M \leftarrow \text{RollOutDynamicModel}(u_M)$
  ▷ **Compute cost functions for each strategy:**
  for $s = 1$ **to** S **in parallel do**
    for $j = 1$ **to** M **do**
      $\text{SubCost}^{(s)}_j = \sum_{h=1}^{H} \text{subcost}^{(s)}(u_{j,h}, \hat{x}_{j,h})$
    end for
  end for
  for π **in** $[\pi^*, \{\pi^{(n)}\}_{n=1}^N]$ **do**
    $\text{Cost}^{\pi}(\hat{\boldsymbol{x}}_M) = \sum_{s=1}^{S} w_{\pi}^{(s)} \text{SubCost}^{(s)}(\hat{\boldsymbol{x}}_M)$
  end for
  ▷ **Evaluate strategies and update distributions:**
  for $n = 1$ **to** N **do**
    $w = \text{softmax}(-\frac{1}{\lambda}\text{Cost}^{\pi^{(n)}})$ ▷(3)
    $\widehat{\boldsymbol{\mu}}_{\pi^{(n)}}, \widehat{\boldsymbol{\Sigma}}_{\pi^{(n)}} = \text{UpdateDistribution}(w, \boldsymbol{u}_M)$ ▷(4)
    ▷ *Select top-K trajectories and assign weights:*
    $\text{topK_index} = \text{argsort}_j(-\text{Cost}^{\pi^{(n)}}(\hat{\boldsymbol{x}}_j))[:topK]$
    $\hat{\boldsymbol{x}}_{\text{topK}} = \hat{\boldsymbol{x}}_M(\text{topK_index})$
    $\boldsymbol{w}_{\text{topK}} = \text{softmax}_{\beta}(\hat{\boldsymbol{x}}_{\text{topK}})$ ▷(15)
    ▷ *Evaluate using Judge strategy:*
    $V^{\pi^{(n)}} = \mathbb{E}_{\boldsymbol{w}_{\text{topK}}}[\text{Cost}^{\pi^*}(\hat{\boldsymbol{x}}_{\text{topK}})]$ ▷(16)
  end for
  ▷ **Update rules:**
  for $n = 1$ **to** N **do**
    $w^{(n)} = \text{softmax}(-\frac{1}{\alpha}V^{\pi^{(n)}})$ ▷(11)
  end for
  $\widehat{\boldsymbol{\mu}}_t \leftarrow (1-\alpha_\mu)\widehat{\boldsymbol{\mu}}_{t-1} + \alpha_\mu \sum_{n=1}^{N} w^{(n)} \widehat{\boldsymbol{\mu}}_{\pi^{(n)},t}$
  $\widehat{\boldsymbol{\Sigma}}_t \leftarrow (1-\alpha_\Sigma)\widehat{\boldsymbol{\Sigma}}_{t-1} + \alpha_\Sigma \sum_{n=1}^{N} w^{(n)} \widehat{\boldsymbol{\Sigma}}_{\pi^{(n)},t}$ ▷(10)
  $\text{SendToActuators}(\widehat{\mu}_{t,1})$
  for $h = 2$ **to** H **do**
    $\widehat{\mu}_{t,h-1} \leftarrow \widehat{\mu}_{t,h}, \quad \widehat{\Sigma}_{t,h-1} \leftarrow \widehat{\Sigma}_{t,h}$
  end for
end while

using central finite differencing

$$\text{Grad}(x) = \frac{\partial d}{\partial x_r}(x) \approx \frac{d(x + \Delta \mathbf{e}_r) - d(x - \Delta \mathbf{e}_r)}{2\Delta} \tag{22}$$

where Δ represents a small increment and $\mathbf{e}_r$ is the corresponding basis vector.

2) Robot Body Representation: For collision detection of robotic manipulators using SDF, the robot's geometry is typically approximated by a set of overlapping spheres. For simplification, these spheres can be described by a set of key points, each with a constant radius r. Each sphere is tied to a corresponding link of the robot. For an L-link robot arm, the local coordinates of the spheres for link_k, denoted as pts'_k, where $k \in [1, L]$, are transformed to workspace coordinates pts_k using the forward kinematic model

$$\text{pts}_k = T_{\text{link}_k \to \text{base}} \cdot \text{pts}'_k, \quad k \in [1, L] \tag{23}$$

where $T_{\text{link}_k \to \text{base}}$ represents the transformation matrix from link_k coordinates to base coordinates. This representation simplifies the collision detection process to a three-dimensional collision query involving the spheres. Specifically, it checks if the SDF value at any key point pt is less than the threshold $\sigma_1 = r + \epsilon$, where ϵ serves as a safety margin.

Given M batches of joint position trajectories $\{q_{i,h}\}_{i=1\ldots M}^{h=1\ldots H}$, the workspace trajectories of the key point $\{pt_{i,h}\}_{i=1\ldots M}^{h=1\ldots H}$ are computed using the kinematic model. Let $\mathbf{P}_i = [\text{Pos}_{i,1}, \text{Pos}_{i,2}, \ldots, \text{Pos}_{i,H}]^T$ represent the positions of key point at H timesteps, the corresponding velocities $\mathbf{V}_i = [\text{Vel}_{i,1}, \text{Vel}_{i,2}, \ldots, \text{Vel}_{i,H}]^T$ can be obtained using a differentiation matrix

$$\mathbf{V}_i = \begin{bmatrix} -1 & 1 & \cdots & 0 & 0 \\ 0 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & -1 & 1 \\ 0 & 0 & \cdots & -1 & 1 \end{bmatrix} \mathbf{P}_i. \tag{24}$$

Note that the subscripts i on $\mathbf{P}_i$ and $\mathbf{V}_i$ represents the trajectory of the ith batch out of M batches.

3) GVM-SDF Cost Function: The proposed method dynamically adjusts the SDF cost function based on the robot's current velocity and the SDF gradient, addressing the issues observed in previous works (see Section II-C). Specifically, the SDF cost function in RAMP [3], denoted by Coll_p, can lead to rapid passage through obstacles, while in STOMP [24] (Coll_{pv}), it can cause the robot to get stuck near obstacles. We assume that the gradient represents the steepest descent direction in the SDF map, as shown in Fig. 6. If the robot's velocity and gradient directions are aligned, indicating the robot is moving away from obstacles, we reduce cost to encourage this motion and prevent stopping near obstacles. Conversely, if the velocity and gradient directions are misaligned, indicating movement toward obstacles, the cost is increased to discourage this behavior and prevent rapid passage through obstacles merely to minimize accumulated cost.

At a given robot state $\hat{x}_{i,h}$, let pt^*_k denote the key point with the minimum SDF value among a set of key points tied to link_k, where $k \in [1, L]$. Its velocity $\text{Vel}(\text{pt}^*_k)$ can be obtained from (24). Precomputed maps are used to obtain $d(\text{pt}^*_k)$, $\Phi(\text{pt}^*_k)$, and $\text{Grad}(\text{pt}^*_k)$. The SDF cost function design methods, including Coll_p, Coll_{pv}, and the proposed GVM-SDF cost function

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

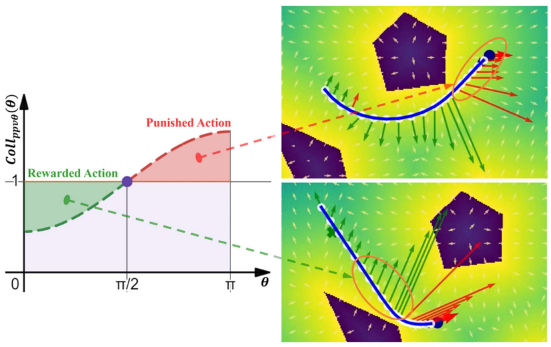


Fig. 6. GVM-SDF: The planned path (blue line) over 20 steps (white points). Arrows indicate gradients, with length representing potential values. Green arrows denote robot velocity aligned with the SDF gradient, corresponding actions should be encouraged. Red arrows indicate the opposite.

$\text{Coll}_{\text{ppv}\theta}$, are defined as follows:

$$\begin{aligned}
\text{Coll}_{p}(\hat{x}_{i,h}) &= \sum\nolimits_{k=1}^{L} \Phi(\text{pt}_k^*) \\
\text{Coll}_{\text{pv}}(\hat{x}_{i,h}) &= \sum\nolimits_{k=1}^{L} \Phi(\text{pt}_k^*) \cdot \|\text{Vel}(\text{pt}_k^*)\| \\
\text{Coll}_{\text{ppv}\theta}(\hat{x}_{i,h}) &= \sum\nolimits_{k=1}^{L} \Big(\Phi(\text{pt}_k^*) \\
&\quad + \Phi(\text{pt}_k^*) \cdot \|\text{Vel}(\text{pt}_k^*)\| \cdot (1 - \rho\cos(\theta)) \Big)
\end{aligned} \tag{25}$$

where

$$\begin{aligned}
\text{pt}_k^* &= \text{argmin}_{pt_k \in \text{key points}(\text{link}_k)} d(\text{pt}_k) \\
\theta &= \cos^{-1}\left(\frac{\text{Grad}(\text{pt}_k^*) \cdot \text{Vel}(\text{pt}_k^*)}{|\text{Grad}(\text{pt}_k^*)||\text{Vel}(\text{pt}_k^*)|} \right).
\end{aligned}$$

Note that θ is the angle between the two vectors $\text{Vel}(\text{pt}_k^*)$ and $\text{Grad}(\text{pt}_k^*)$. If $\theta < \pi/2$, it indicates that the two vectors are aligned. The parameter $\rho \in [0, 1]$ controls the degree of encouragement and penalty.

B. Using the TRAC-IK for Guiding MPPI

We deviate from traditional methods [3], [9] that rely on manipulability indices for mitigating singularities in cost function design, as they tend to be overly complex and yield suboptimal outcomes. Instead, we employ multiprocessing using the TRAC-IK to guide MPPI out of local minima and simultaneously optimize its operability. This method continuously generates IK solutions $q_{\text{des},t}$ based on the current joint position q_t and target Cartesian pose $X_g(\mathbf{p}_g, \mathbf{R}_g)$. Given robot state $\hat{x}_{i,h}$ and joint state $q_{i,h}$, the corresponding joint-space goal cost $\text{dist}_{\text{joint}}$ can be designed as

$$\begin{aligned}
q_{\text{des},t} &= \text{TRAC-IK}(q_t, X_g) \\
\text{dist}_{\text{joint}}(\hat{x}_{i,h}) &= \|q_{i,h} - q_{\text{des},t}\|_2 .
\end{aligned} \tag{26}$$

In cases where the TRAC-IK is infeasible or the target pose $X_g(\mathbf{p}_g, \mathbf{R}_g)$ is outside the robot's workspace and given the robot EE pose $X_{i,h}^{\text{ee}}(\mathbf{p}_{\text{ee}}, \mathbf{R}_{\text{ee}})$ at robot state $\hat{x}_{i,h}$, we design a Cartesian-space goal cost $\text{dist}_{\text{cart}}$

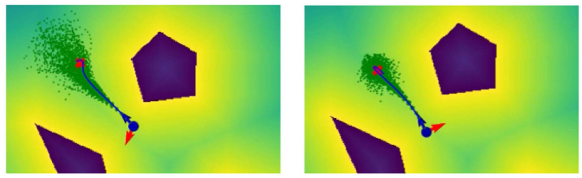
$$\begin{aligned}
\text{dist}_{\text{cart}}(\hat{x}_{i,h}) &= \alpha_1 \|\mathbf{p}_{ee} - \mathbf{p}_g\|_2 \\
&\quad + \alpha_2 \cos^{-1}\left(2\langle \hat{\mathbf{q}}_{ee}, \hat{\mathbf{q}}_g \rangle^2 - 1\right)
\end{aligned} \tag{27}$$

where $\mathbf{p}_{\text{ee}}, \mathbf{p}_g$ are the position vectors and $\hat{\mathbf{q}}_{\text{ee}}, \hat{\mathbf{q}}_g$ are the corresponding quaternions representing the rotation matrices $\mathbf{R}_{\text{ee}}, \mathbf{R}_g$. α_1 and α_2 are weighting coefficients that control the relative importance of position error and orientation error, respectively.

While the Euclidean norms $\|.\|_2$ in $\text{dist}_{\text{joint}}$ and $\text{dist}_{\text{cart}}$ drive the robot toward the target pose, they exhibit low sensitivity near the target pose, potentially leading to significant errors during approach. In this regard, we integrate sparse reward as one cost function to minimize end-pose error

$$R_s(\hat{x}_{i,h}) = 1 - \exp\left(-\text{dist}(\hat{x}_{i,h})^2 / \left(2\xi^2\right)\right) \tag{28}$$

where dist defaults to $\text{dist}_{\text{joint}}$, and switches to $\text{dist}_{\text{cart}}$ when the TRAC-IK fails to return results. ξ is the convergence radius. When dist is within the radius ξ, SparseReward rapidly decreases costs, driving sampled trajectories to quickly converge on the target region, whereas outside the radius, it has no guiding effect. This cost modulation approach allows the robot to automatically slow down near the target location and reduce reach errors. Fig. 7 presents a comparison of the MPPI-sampled trajectories with and without the sparse reward term. The results show that using the sparse reward term, the MPPI converges faster to the target pose.



(a) (b)

Fig. 7. Sparse reward illustration. (a) Without $R_s(\text{dist})$. (b) With $R_s(\text{dist})$.

In addition, robot dynamics constraints $C_{\text{safe}}(\hat{x}_{i,h})$ are designed to ensure the safety during motion, employing the approach from STORM [9], which provides a specific implementation of MPPI for robotic manipulator. $C_{\text{safe}}(\hat{x}_{i,h})$ weights together joint position limits C_{bound}, joint velocity constraint C_{stop}, and self-collision avoidance $C_{\text{self-}coll}$, utilizing a neural network to accelerate self-collision queries.

C. Cost Function Design for Different Strategies

The cost function for each MPPI planner combines multiple subcost functions weighted together based on their respective strategies, sharing a unified framework

$$\begin{aligned}
\text{Cost}^{\pi}(\hat{\boldsymbol{x}}_i) &= \sum\nolimits_{h=1}^{H} w_g \text{dist}(\hat{x}_{i,h}) + w_c \text{Coll}(\hat{x}_{i,h}) \\
&\quad + w_r R_s(\hat{x}_{i,h}) + w_s C_{\text{safe}}(\hat{x}_{i,h})
\end{aligned} \tag{29}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
WEIGHT ALLOCATION FOR STRATEGIES IN PMPPI AND MPPI-BASELINE

System	weight	π_{greedy}	π_{sensi}	π_{Judge}	MPPI-Baseline
2-D Point	$\mathbf{w_g}$	40.0	5.0	10.0	10.0
	$\mathbf{w_c}$	1.0	3.0	2.0	2.0
	$\mathbf{w_r}$	20.0	0.0	1.0	1.0
Franka	$\mathbf{w_g}$	80.0	10.0	10.0	10.0
	$\mathbf{w_c}$	80.0	250.0	80.0	80.0
	$\mathbf{w_r}$	200.0	0.0	80.0	80.0

where $\text{Cost}^{\pi}(\hat{\boldsymbol{x}}_i)$ is ith trajectory's cost on different strategies, $i \in [1, M]$. dist takes value from $\text{dist}_{\text{joint}}$ by default, switching to $\text{dist}_{\text{cart}}$ in case TRAC-IK fails to yield results. Collision term Coll is chosen from Coll_p, Coll_{pv}, or $\text{Coll}_{\text{ppv}\theta}$. The terms w_g, w_c, and w_r denote the weights for the goal-reaching cost, collision penalty, and sparse reward, respectively.

In the PMPPI framework, different strategies are distinguished by their cost function formulations, each emphasizing specific aspects of trajectory planning.

1) *Greedy Strategy (π_{greedy})*: The greedy strategy prioritizes rapid target reaching with reduced sensitivity to obstacles. It assigns larger weights to goal-related cost terms, such as dist and R_s, to encourage fast convergence of the planning path to the target pose. To manage obstacle avoidance, it incorporates Coll_p, which balances efficiency and Safety.
2) *Sensitive Strategy (π_{sensi})*: The sensitive strategy focuses on collision avoidance, exhibiting heightened responsiveness to obstacles. It assigns higher weights to collision-related cost terms, utilizing $\text{Coll}_{\text{ppv}\theta}$ to enhance its sensitivity to the environment. To improve exploration in obstacle-dense regions, the goal-related cost term R_s is omitted, allowing the planner to prioritize safety and adaptability.
3) *Judge Strategy (π_{Judge})*: The Judge strategy π^*, distinct from each MPPI planner's own planning strategy (e.g., π_{greedy}, π_{sensi}), does not participate in generating planning path step, solely evaluating submitted trajectories based on their distance to target pose (e.g., dist, R_s) and risk of collisions with environment (Coll_p) in dynamic obstacle task. It omits dynamic constraints of the robot to ensure fairness and generality in trajectory evaluation. Its associated weights are calibrated between those of the greedy and sensitive strategies, as shown in Table I.

The weights assigned to each cost term for these strategies are systematically listed in Table I to highlight their respective priorities and distinctions. These parameters will be detailed in Section VI-B to ensure a fair and robust comparison between PMPPI and MPPI-Baseline.

VI. SIMULATION

The performances of PMPPI have been simulated and evaluated against the standard MPPI across four simulation scenarios: a 2-D PointMass and a 7-DoF Franka robotic manipulator, each encountering obstacles moving vertically and horizontally. For comparison purpose, performance of the proposed GVM-SDF Cost with the traditional collision cost function was evaluated too.

1) *2-D PointMass (Task 1–2)*: These tasks involve a 2-D PointMass double integrator model tasked with achieving bidirectional motion between two waypoints over 20 laps. The control inputs consist of acceleration $[a_x, a_y]$ in the x and y directions, respectively, with maximum acceleration limits set to 0.10 m/s^2. Integration of these inputs yields velocity and position states $[V_x, V_y, P_x, P_y]$. The 2-D environment is represented by a 378×378-pixel grayscale image, where 0-value pixels denote obstacles, normalized, and abstracted into a 1-meter motion area. In Task 1, the PointMass must navigate around high-speed obstacles moving vertically, while in Task 2, it faces obstacles moving horizontally. Obstacles move at a speed of 0.133 m/s, with a maximum displacement of 0.079 m from their initial position. A collision occurs if the PointMass's current SDF Potential value exceeds 0.90 ($\Phi(x) > 0.90$). A total of 360 sampled trajectories with a horizon of 20 are sampled from the final mixed Gaussian distribution and that of π_{greedy}, π_{sensi} MPPI planners', with 80 trajectories for each planner. The top-K is set to 20, and the temperature parameters in PMPPI are $\alpha = \lambda = 1.0$, and $\beta = 0.8$.

2) *Franka (Task 3–4)*: Similar to the previous challenge, Task 3 and Task 4 involve a 7-DoF Franka Emika Panda arm on the Isaac Gym platform [34]. The objective is to maneuver the EE between two target poses over 10 laps without colliding with dynamic obstacles. The system receives joint accelerations $\ddot{q} = [\ddot{q}_1, \ddot{q}_2, \ldots, \ddot{q}_7]$ as input, with a maximum joint acceleration limit of 0.5 m/s^2. These inputs are integrated to compute joint velocity and position state $[\dot{q}, q]$. The obstacle is a cylinder with a radius of 0.20 m and a height of 0.51 m. In Task 3, obstacles move vertically within a range of 0.4 m to 0.8 m along the z-axis. Whereas in Task 4, they move horizontally within a range of −0.4 to 0.4 m along the y-axis. Obstacle velocity is specified as 0.269 m/s. A collision is identified if the SDF Potential at any key point pt exceeds 0.90 ($\Phi(\text{pt}) > 0.90$). We sample 580 trajectories of 30 timesteps, with 80 trajectories from each MPPI planner. The PMPPI parameters match the 2-D PointMass setting.

A. Visualization of the PMPPI Performance

The PMPPI algorithm's adaptive behaviors in dynamic obstacle avoidance for 2-D PointMass and Franka simulations are visualized in Fig. 8, with the motion state during this process also recorded. Fig. 8(d) depicts the movements of Franka's joint 1 and joint 5, with the target joint positions output by the TRAC-IK represented with dashed lines. As the robot approaches an obstacle, the mixed planned path (blue) tends to align with that of π_{sensi} (green), with the weight of π_{sensi} being larger than that of π_{greedy} in this process. Conversely, when the robot is further away from obstacles, the mixed planned path transitions to that of π_{greedy} (red), and the weight's value of π_{greedy} increases toward one. A smooth trajectory is generated throughout the dynamic obstacle avoidance process.

B. Trajectory Analysis

The algorithm's performance is evaluated against other algorithms, including STORM [9], QP-based method NEO [35], and $\text{MPPI-Coll}_{\text{ppv}\theta}$. Metrics in the comparison are: the time

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

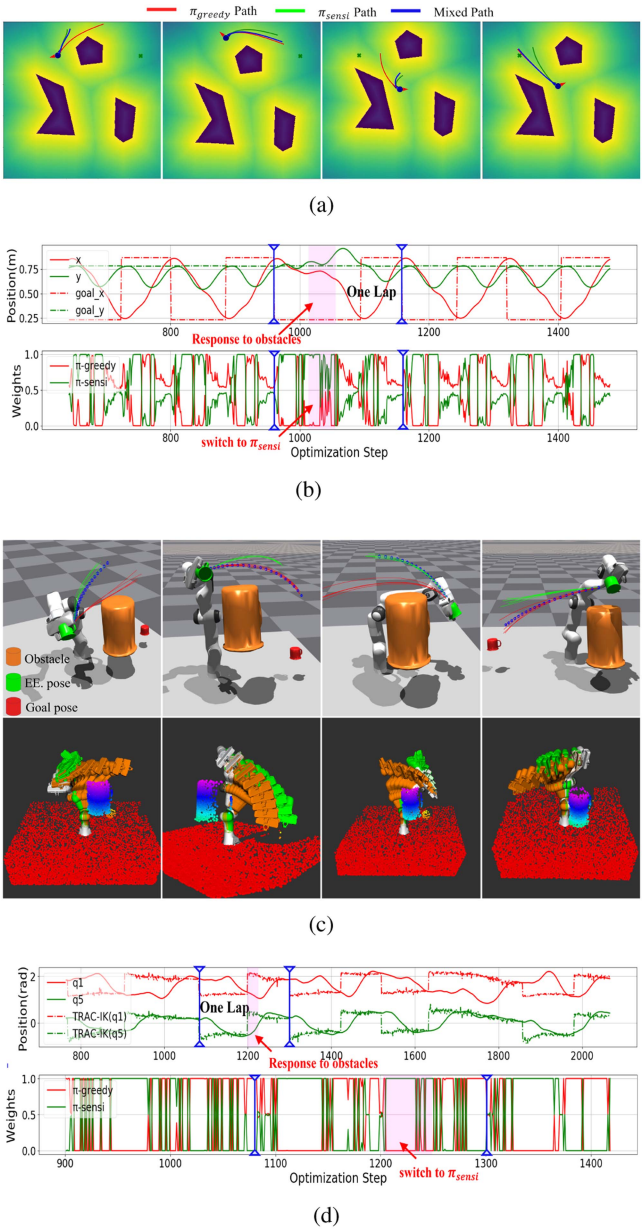


Fig. 8. Illustration of the PMPPI's performance over one lap for 2-D PointMass (Task 1) and Franka (Task 3), with recorded positions and weight distributions. Bottom row of (c) are environment point clouds and joints sequences, with orange and green trajectories representing π_{greedy} and π_{sensi} plans. (a) PMPPI performance (one lap) for 2-D PointMass. (b) Positions and weight distribution for 2-D PointMass. (c) PMPPI performance (one lap) for Franka. (d) Positions and weight distribution for Franka.

taken to complete task (Running time), the total number of iterations or optimization steps (Opt-steps), the length of the path traveled (Path length), the total number of collisions (Collisions), the likelihood of a collision occurring in each lap (Crash rate), and the robot average velocity (Avg.Vel.). For the Franka arm, the path length represents the cumulative length of all joint paths and the Avg.Vel. denotes the average speed of the EE.

As the algorithmic structures of the PMPPI and the MPPI differ, comparison results may become ambiguous under different comparison rules. Therefore, we conducted a grid search and

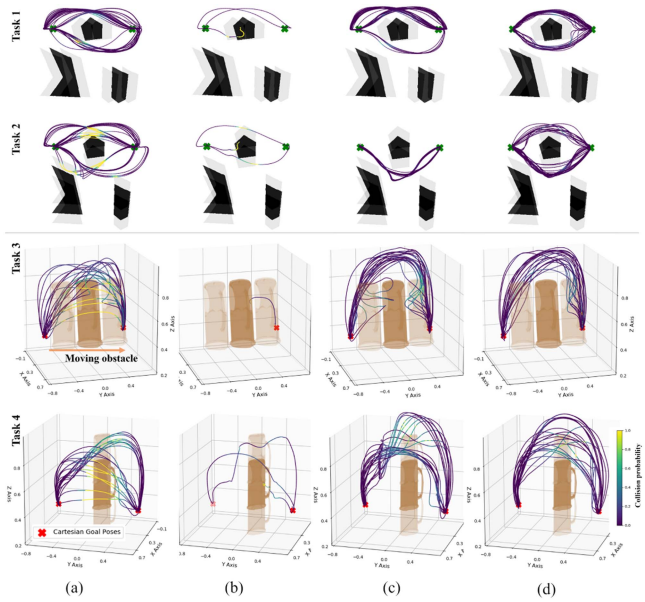


Fig. 9. Comparison of trajectories generated by the MPPI, utilizing Coll_p, Coll_{pv}, and $\text{Coll}_{\text{ppv}\theta}$, and the PMPPI. (a) Coll_p. (b) Coll_{pv}. (c) $\text{Coll}_{\text{ppv}\theta}$. (d) PMPPI.

selected a specific pair of weights (w_g, w_c) that yielded better experimental results for the MPPI, aiming for a more persuasive comparison with the PMPPI. Table I outlines the specific weight values used. The detailed implementation of this selection process is described in Section VI-C. The MPPI-baseline achieves the target task quickly while ensuring path safety with this pair of weights. π_{Judge} in the PMPPI is also aimed at achieving a similar balance. The weights of π_{Judge} in the PMPPI with this optimal pair and other settings are aligned for both algorithms.

1) PMPPI Versus MPPI With Distinct SDF Cost Functions: Fig. 9 displays trajectories generated by the PMPPI and MPPI-baseline with different SDF cost functions across four tasks, with translucent images representing the reciprocating motion of obstacles and their motion boundaries. The trajectories of a 2-D PointMass on a 2-D plane are shown in the first two rows, while the last two rows display the Franka EE trajectories in the workspace. Fig. 9(a)–(c) is trajectories generated by the MPPI, each utilizing a distinct SDF cost function: Coll_p from RAMP [3], Coll_{pv} from STOMP [24], and $\text{Coll}_{\text{ppv}\theta}$ from our proposed GVM-SDF approach. Fig. 9(d) shows the trajectories generated by the PMPPI. Specific evaluation results can be found in Table II.

Upon comparing Fig. 9(a)–(c), it is observed that trajectories generated by the MPPI-baseline using Coll_p tend to pass through obstacles multiple times, with a high collision count, suggesting a rapid traversal to minimize cost. Those trajectories generated using Coll_{pv} often terminate near obstacle areas, frequently failing tasks (e.g., Task 1–4 marked as failed due to local minima), implying a tendency to reduce speed to zero near obstacles. In contrast, the proposed $\text{Coll}_{\text{ppv}\theta}$ formulation significantly improves safety, reducing collision rates to 1.875% in Task 1 with only 2 collisions. However, this conservative

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MPPI-BASELINE WITH DIFFERENT STRATEGIES VERSUS MPPI WITH DISTINCT SDF COST FUNCTIONS VERSUS PMPPI

SYS.	Task	Controller	Running Time (s)	Opt-Steps	Collisions	Crash rate (%)	Path length[4]	Avg. Vel.(m/s)
2-D Point Mass[2]	1	MPPI-π_{greedy}[1]	159.866	2804	202	69.58	34.144	0.143
		MPPI-π_{sensi}	183.175	3213	5	9.583	34.288	0.125
		MPPI-Coll$_p$[1]	172.596	3028	73	47.5	35.653	0.207
		MPPI-Coll$_{pv}$	—[3]	—	—	—	—	—
		MPPI-Coll$_{ppv\theta}$	181.559	3185.25	**2**	**1.875**	33.515	**0.185**
		PMPPI	**178.783**	**3004.75**	8.25	6.25	**30.18**	0.169
	2	MPPI-π_{greedy}	152.0	2666	182.833	90.833	33.253	0.147
		MPPI-π_{sensi}	162.136	2844	0	0	33.602	0.139
		MPPI-Coll$_p$	163.020	2860	13	15	34.632	0.213
		MPPI-Coll$_{pv}$	—	—	—	—	—	—
		MPPI-Coll$_{ppv\theta}$	162.146	2844.667	1.333	4.375	34.548	**0.212**
		PMPPI	**160.144**	**2691.5**	**0**	**0**	**28.152**	0.176
Franka[2]	3	MPPI-π_{greedy}	69.879	1281	108	87.5	143.135	0.413
		MPPI-π_{sensi}	151.444	2777	0.667	5	198.739	0.266
		MPPI-Coll$_p$	91.482	1677.333	54.75	46.25	157.726	0.462
		MPPI-Coll$_{pv}$	—	—	—	—	—	—
		MPPI-Coll$_{ppv\theta}$	120.179	2203.5	4.75	3.333	185.018	0.396
		PMPPI	**110.301**	**1978.5**	**0**	**0**	**175.434**	**0.416**
	4	MPPI-π_{greedy}	70.902	1300	102.75	92.5	141.514	0.4
		MPPI-π_{sensi}	188.463	3455	**0**	**0**	226.894	0.254
		MPPI-Coll$_p$	91.804	1683.25	47.5	41.667	160.022	0.475
		MPPI-Coll$_{pv}$	—	—	—	—	—	—
		MPPI-Coll$_{ppv\theta}$	146.222	2681	1	5	205.094	0.365
		PMPPI	**133.925**	**2402.25**	3.5	8.75	**189.38**	**0.369**

[1] The MPPI-Coll$_p$ and MPPI-π_{greedy} methods proved ineffective due to its excessive collision frequency, despite its lower running time.
[2] Obstacle Speed: 0.133 m/s (2-D PointMass), 0.269 m/s (Franka)
[3] — Task Failed (Algorithm Trapped in Local Minimum)
[4] Units: m (2-D PointMass), rad (Franka)

strategy increases path lengths by 13.6% compared to PMPPI (33.515 m versus 30.18 m) and introduces trajectory oscillations visible in Fig. 9(c). Comparing these figures with Fig. 9(d), and data from Table II, trajectories generated by the PMPPI exhibit superior safety guarantees, shorter trajectory lengths, and smoother motion performance compared to the various MPPI schemes.

2) PMPPI Versus MPPI With Various Strategies: The differences between the MPPI planners used in the PMPPI framework highlight PMPPI's ability to effectively integrate multiple strategies. It is seen from Table II that the trajectories generated by the MPPI with the greedy strategy tend to pass through obstacles multiple times, prioritizing rapid target reaching with less sensitivity to obstacles. In contrast, the trajectories generated by the sensitive MPPI planner exhibit heightened responsiveness to obstacles, but result in longer, more time-consuming, and winding paths.

3) SDF-Based Versus Voxel-Based Cost Function: To demonstrate the superiority of the SDF-based environmental cost function over the discrete cost function, which is based solely on collision occurrence [9], [23], we present experimental results from STORM [9]. STORM employs SceneCollisionNet [23] for collision prediction, which constructs its discrete environmental cost function using a voxel-grid-based collision detection method. Specifically, the collision detection function determines if a robot link point cloud pc_l collides with the environment point cloud pc_{env} based on the robot link's pose X_l, using the following formula:

$$\text{Coll}(\text{pc}_l, \text{pc}_{\text{env}}, X_l) = \begin{cases} 1, & \text{if collision} \\ 0, & \text{otherwise.} \end{cases} \tag{30}$$

TABLE III
PMPPI VERSUS STORM

SYS.	Task	Controller	Running Time (s)	Opt-Steps	Collisions	Crash rate (%)	Path length	Avg. Vel.(m/s)
Franka	3	STORM [9]	**90.005**	**1650.25**	168.5	80	**161.476**	**0.459**
		PMPPI	110.301	1978.5	**0**	**0**	175.434	0.416
	4	STORM [9]	**98.990**	**1815**	251.25	93.75	**168.602**	**0.438**
		PMPPI	133.925	2402.25	**3.5**	**8.75**	189.38	0.369

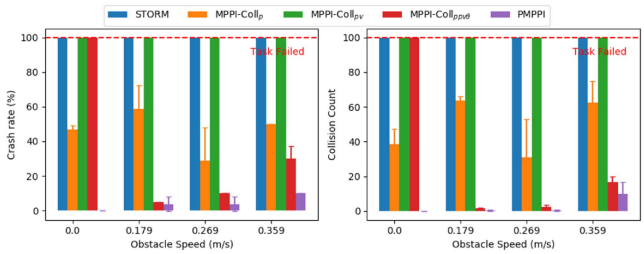


Fig. 10. Different obstacle speed test on Franka (Task 3).

While STORM originally applied this method to static environments, we extend it to handle dynamic environments. Similar to the perception module in PMPPI, the updated approach processes real-time point cloud data at 30 Hz to dynamically construct and update the voxel grid, enabling STORM to react to dynamic obstacles while maintaining the computational efficiency of voxel-grid-based collision detection. This real-time processing capability ensures a fair comparison with the PMPPI method. As shown in Table III, STORM's discrete voxel grid method results in a higher collision frequency and lower sensitivity to dynamic obstacles compared to both the previous MPPI method with distinct SDF cost functions and the PMPPI method. This is due to the fact that STORM's discrete collision cost, based on binary classification, does not capture the fine-grained environmental changes as effectively as the continuous SDF-based cost function methods, which can provide a more nuanced understanding of dynamic obstacles.

4) Tests on Different Obstacle Speed: We vary obstacle velocities from 0.0 to 0.359 m/s to disrupt Franka's motion in Task 3. Franka's collisions and crash rate under these velocities remain consistent with those described earlier (see Fig. 10). MPPI using Coll_{pv} fails across all velocities, while the MPPI using Coll_p shows high values in both performance metrics with large variances. In contrast, the MPPI using $\text{Coll}_{\text{ppv}\theta}$ performs poorly at the static scenario but effectively reduces collisions and crash rate at other velocities. The proposed PMPPI approach outperforms all the MPPI methods, achieving superior collisions and crash rate across varying obstacle velocities.

C. Distribution Analysis

In this work, a grid search is conducted, with the purpose to compare the performance of the PMPPI and the MPPI and address concerns regarding biased evaluations resulting from intentionally selecting suboptimal weight parameters for the MPPI. This involved perturbing the cost weights (w_g, w_c) of the MPPI and the Judge strategy in the PMPPI while keeping other weights or tunable parameters unchanged. It is worth noting that

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
GRID SEARCH PARAMETERS

System	Weight	Lower Bound	Upper Bound	Interval
2-D Point	w_g	5	40	5
	w_c	1	3	1
Franka	w_g	10	80	10
	w_c	80	250	10

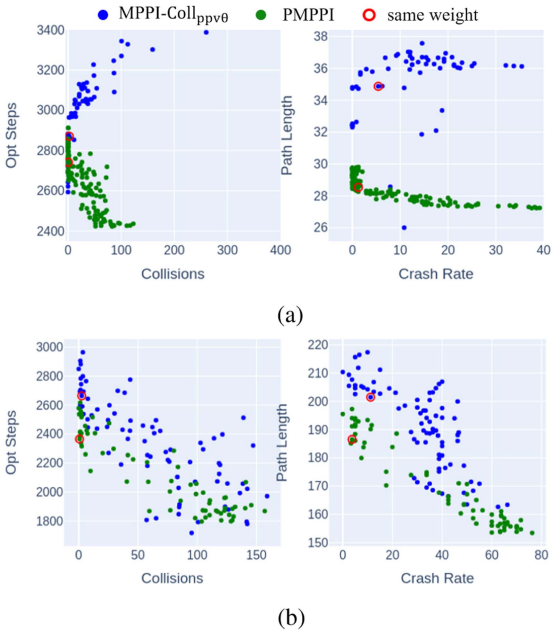


Fig. 11. Running time and collisions distribution, as well as path length and crash rate distribution for both algorithms in Task 1 and Task 3, respectively. (a) 2-D PointMass. (b) Franka.

MPPI adopts the SDF cost function $\text{Coll}_{\text{ppv}\theta}$, which has shown better performance compared to other cost function methods (see Section VI-B for details). Table IV outlines the grid search boundary, with the interval column specifying the search step. Fig. 11 displays scatter plots of the grid search results, with blue points representing distribution results from the MPPI and green points representing those from the PMPPI. In both systems, the distributions of time consumption metrics (running time and path length) for PMPPI, compared to those for the MPPI, are more concentrated toward the bottom, indicating that the PMPPI achieves tasks in less time and with shorter path lengths. Similarly, the distributions of safety metrics (collisions and crash rate) for the PMPPI, relative to the MPPI, are skewed toward the lower-left corner, suggesting fewer collision occurrences and enhanced safety with the PMPPI. The red circles in Fig. 11 mark the results from the MPPI and the PMPPI using the same weight pair. Detailed analysis of these results has been discussed in Section VI-B.

D. Comparison With Non-MPPI Methods

A comparative analysis of the PMPPI method against the QP-based NEO algorithm [35] was performed in dynamic obstacle avoidance tasks, where reactive approaches consistently outperform traditional global planning methods in unpredictable environments characterized by rapid changes and unforeseen interruptions. The evaluation encompassed PMPPI, various MPPI variants, and NEO, with the latter utilizing quadratic programming enhanced by slack variables to balance collision avoidance and trajectory optimization, as implemented through Reflexxes Motion Libraries [36]. Tests were conducted under conditions, with obstacles moving vertically or laterally at speeds ranging from 0.0 to 0.359 m/s along the robot arm's predefined path, as specified in Task 3 settings designed to simulate real-world dynamic interference. Results listed in Table V reveal that NEO struggled significantly with local minima, a limitation stemming from its reactive optimization being overly constrained by immediate obstacle proximity, leading to a 45% crash rate and a shorter yet riskier EE path of 26.515 m at 0.269 m/s, with complete task failure observed under static conditions due to its inability to adapt beyond initial constraints.

TABLE V
MPPI-BASED VERSUS QP-BASED CONTROLLERS IN DYNAMIC OBSTACLE ENVIRONMENT (TASK3)

Test	Obs. Vel. (m/s)	Method	Crash (%)	EE Path (m)	Joint Path (rad)	Running Time (s)
1	0	STORM	98.33	**42.533**	176.336	135.66
		MPPI-Coll$_p$	46.67	43.388	**174.653**	148.39
		MPPI-Coll$_{\text{pv}}$	—[1]	—	—	—
		MPPI-Coll$_{\text{ppv}\theta}$	20.00	44.011	185.792	163.50
		PMPPI	**0.00**	46.165	178.914	**125.20**
		QP-based (NEO)	—	—	—	—
2	0.179	STORM	93.75	40.179	159.573	87.81
		MPPI-Coll$_p$	58.75	39.574	**150.414**	**87.47**
		MPPI-Coll$_{\text{pv}}$	—	—	—	—
		MPPI-Coll$_{\text{ppv}\theta}$	5.00	51.274	201.495	151.46
		PMPPI	**3.75**	48.089	186.772	134.24
		QP-based (NEO)	65.00	**35.182**	154.069	177.38
3	0.269	STORM	90.00	42.498	165.326	**98.32**
		MPPI-Coll$_p$	28.75	46.518	171.231	106.45
		MPPI-Coll$_{\text{pv}}$	—	—	—	—
		MPPI-Coll$_{\text{ppv}\theta}$	10.00	54.461	207.318	154.34
		PMPPI	**3.75**	48.116	185.273	131.02
		QP-based (NEO)	45.00	**26.515**	**99.293**	143.33
4	0.359	STORM	96.25	41.643	161.627	**91.18**
		MPPI-Coll$_p$	50.00	44.024	162.880	98.43
		MPPI-Coll$_{\text{pv}}$	—	—	—	—
		MPPI-Coll$_{\text{ppv}\theta}$	30.00	52.384	199.208	140.50
		PMPPI	**10.00**	48.468	188.315	136.02
		QP-based (NEO)	50.00	**26.851**	**101.599**	138.53

[1] — indicates task failure.
[2] Bold values denote optimal metrics per test: lowest crash rate, shortest EE/joint path, and fastest computation time.

In contrast, PMPPI achieved a markedly lower crash rate of 3.75% and maintained a safer path length of 48.116 m at the same speed. This was accomplished through the use of wider-ranging trajectory sampling, which proved effective in obstacle avoidance. By contrast, NEO's strategy of relying on constrained lateral shifts-essentially a reactive sidestepping approach-led to a significantly higher crash rate of 45%. This highlights NEO's vulnerability to collisions when the motion of obstacles restricts its available maneuverable space This distinction highlights PMPPI's proactive adaptability: its sampling-based approach explores a richer set of potential trajectories, enabling it to anticipate and circumvent obstacles rather than merely respond to them, unlike NEO's more rigid framework, which falters when faced with complex spatial demands. Although NEO produced a shorter joint path (99.293 rad versus PMPPI's 185.273 rad), reflecting its preference for minimal adjustments, this efficiency came at the cost of prolonged stalling near obstacles, inflating

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

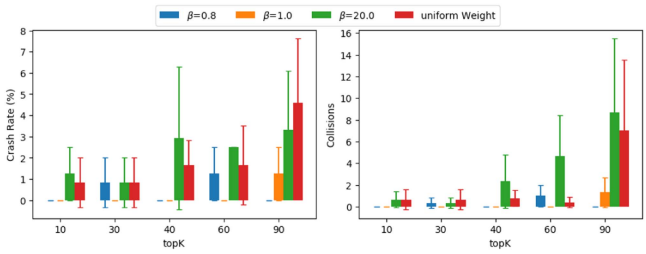


Fig. 12. Analysis of β in 2-D PointMass.

its running time to 143.33 s compared to PMPPI's 131.02 s. At higher speeds (0.359 m/s), NEO's crash rate escalated to 50%, underscoring its diminishing reliability as dynamic complexity increased, while PMPPI maintained a 10% crash rate and a running time of 136.02 s against NEO's 138.53 s, further evidencing its resilience. These findings, supported by data and experimental videos, highlight PMPPI's superior robustness and efficiency in dynamic scenarios, driven by its ability to generate safer, adaptive trajectories. The broader exploration inherent in PMPPI's design not only mitigates the pitfalls of local minima but also fosters a more versatile navigation strategy, adept at handling the unpredictability of moving obstacles.

E. Softmax Weight Analysis

In Section IV, the softmax weight approach is introduced as an enhancement over uniform weight to more effectively capture differences in strategies. Its efficacy is validated through experimental results presented in Fig. 12.

In Task 1, the PMPPI's performance with softmax weight, using varying β values, is compared against uniform weight, assessing collisions and crash rate across different top-K values via bar chart analysis. Here, top-K represents the number of trajectories each MPPI planner submits to the Judge strategy. The analysis shows that larger top-K values can obscure distinctions among planners, as excessive trajectory overlap dilutes their unique distributions. Without mitigation for this overlap, the Judge strategy produces similar evaluations, exacerbating uniform weight's poor differentiation.

Uniform weight exhibits sharply rising collision metrics and high variance with increasing top-K, reflecting its limited ability to differentiate planners. In contrast, softmax weight consistently yields lower collision metrics across top-K values. Experiments with varying β further reveal that higher values approximate uniform weight, degrading performance, whereas lower β values prioritize low-cost trajectories, mitigating top-K's impact and enhancing strategy differentiation. Thus, β serves as a tunable parameter, enabling the softmax weight to precisely reflect strategy variations, outperforming uniform weight in both accuracy and robustness.

F. Computation Cost Analysis

The computational costs of the PMPPI, MPPI-Coll$_{\text{ppv}\theta}$, and STORM methods across simulation Tasks 3–4 were compared, focusing on computation time and resource consumption (see Table VI). The experimental setup was consistent for all methods, utilizing the same task and computational platform, which included an NVIDIA GeForce RTX 3060 GPU and an 8-core AMD Ryzen 7 5800H CPU. The results show that, due to efficient GPU optimization, the use of the torch library for computational acceleration, and inherent optimizations in the PMPPI algorithm (such as parallel computation and task allocation strategies), PMPPI significantly enhances computational efficiency and reduces computation time while maintaining a balanced resource usage profile. Compared to MPPI-Coll$_{\text{ppv}\theta}$ and STORM, PMPPI demonstrates a negligible increase in computation time (32.25 $\pm$ 0.12ms versus 31.852 $\pm$ 0.26 ms) and shows comparable performance in most resource consumption metrics. While PMPPI does use slightly more system memory, this difference does not have a substantial effect on overall performance. Thus, PMPPI achieves an effective balance between computational efficiency and resource consumption, making it well-suited for real-time computation tasks.

TABLE VI
COMPARISON OF COMPUTATIONAL METRICS FOR PMPPI, MPPI-COLL$_{\text{PPV}\theta}$, AND STORM

Metric	PMPPI	MPPI-Coll$_{\text{ppv}\theta}$	STORM
CPU Usage (%)	22.51 $\pm$ 1.20	29.76 $\pm$ 2.03	**35.24 $\pm$ 4.63**
GPU Usage (%)	67.23 $\pm$ 1.91	**70.57 $\pm$ 2.49**	68.87 $\pm$ 3.66
Memory Used (GB)	**11.35 $\pm$ 0.00**	11.29 $\pm$ 0.02	11.02 $\pm$ 0.06
GPU Memory Used (GB)	3.32 $\pm$ 0.01	**3.44 $\pm$ 0.02**	3.40 $\pm$ 0.01
Planning Time (ms/iter)[1]	**32.25 $\pm$ 0.12**	31.852 $\pm$ 0.19	31.82 $\pm$ 0.23

[1] The "planning time (ms/iter)" refers specifically to the computation time for each optimization iteration of the algorithm, excluding other tasks such as point cloud processing and additional computational overhead.

VII. REAL-WORLD EXPERIMENTS

The PMPPI algorithm was finally applied in real-world experiments conducted to validate the effectiveness of the PMPPI method. These experiments include target-reaching tasks in various environments, dynamic obstacle avoidance, and grasping tasks with human–robot interaction on the Franka Emika Panda.

A. Experimental Setup

The experimental computer operated on the Ubuntu 20.04 software system and was equipped with an NVIDIA GeForce RTX 3060 graphics card and an 8-core AMD Ryzen 7 5800H CPU. An Azure Kinect camera was used to generate workspace point clouds for constructing the SDF Map, with the robot body filter package [37] utilized beforehand to create a mask for filtering out the body of the Franka robot from the point clouds at a frequency of 30 Hz. In addition, an Intel RealSense D435i camera was employed to capture point clouds on the desktop for object detection during grasping tasks. The control parameters, such as the planning horizon H, number of sampled trajectories M, and cost weights, remained consistent with those used in the simulation experiments.

B. Reaching Targets With and Without Obstacles

The experiments were conducted in three distinct scenarios:
1) an obstacle-free desktop;

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
PATH PLANNING ALGORITHMS PERFORMANCE COMPARISON IN REAL ROBOT EXPERIMENTS (MEAN ± STD)

Task	Methods	EE. Path (m)	Joint Path (rad)	Crash rate (%)	Planning time (s/iter)	Running time (s/lap)
Static Obstacle Free	MPPI-Coll$_{ppv\theta}$	2.730 ± 0.002	**4.796 ± 0.038**	0.0 ± 0.0	**0.038 ± 0.000**	5.249 ± 0.191
	PMPPI	2.719 ± 0.003	5.237 ± 0.035	0.0 ± 0.0	0.041 ± 0.000	**5.206 ± 0.080**
	RRTConnect	**2.578 ± 0.000**	5.815 ± 0.050	0.0 ± 0.0	–	5.410 ± 0.029
	RRT-CHOMP	2.639 ± 0.000	5.618 ± 0.000	0.0 ± 0.0	–	5.609 ± 0.011
Static Obstacles	MPPI-Coll$_{ppv\theta}$	3.320 ± 0.006	**5.860 ± 0.025**	**10.0 ± 9.0**	**0.041 ± 0.000**	7.697 ± 0.049
	PMPPI	**3.233 ± 0.007**	5.970 ± 0.014	**10.0 ± 9.0**	0.042 ± 0.000	**6.730 ± 0.057**
	RRTConnect	3.672 ± 0.224	8.421 ± 1.231	33.4 ± 5.5	–	9.006 ± 1.529
	RRT-CHOMP	3.408 ± 0.025	8.674 ± 1.106	29.0 ± 1.0	–	11.134 ± 15.475
Dynamic Obstacle Avoidance	MPPI-Coll$_{ppv\theta}$	3.639 ± 0.036	**6.417 ± 0.178**	30.0 ± 21.0	0.040 ± 0.000	8.397 ± 1.090
	PMPPI	3.737 ± 0.014	6.652 ± 0.023	**10.0 ± 9.0**	0.040 ± 0.000	**7.381 ± 0.062**
	RRTConnect	3.780 ± 0.265	8.602 ± 1.241	–	–	13.582 ± 5.464
	RRT-CHOMP	**3.479 ± 0.043**	8.829 ± 0.663	–	–	17.327 ± 22.423

2) a desktop with two boxes forming a static obstacle stack of 50 cm in total height;
3) dynamic obstacles, where a 70 cm-long rectangular box swings up and down in the workspace.

The Franka robot performed 20 laps of back-and-forth movements between two points in each scenario.

We compared PMPPI with MPPI-Baseline (using GVM-SDF cost, which has been shown to be effective compared to other SDF cost functions), as well as traditional methods based on the MoveIt platform: RRT-Connect (sampling-based) and CHOMP (optimization-based). CHOMP is an optimization method in joint space that tends to get stuck in local minima, especially in the presence of obstacles. To mitigate this, we combine RRT-Connect with CHOMP, where RRT-Connect generates an initial trajectory, which is subsequently optimized by CHOMP for smoother and safer paths. In our experiments, real-world tests were conducted using pure point cloud data to ensure a fair comparison. The MPPI-based methods build an SDF cost map from the point cloud data in real time, while MoveIt constructs an Octomap (occupancy map) from the same data, updating at 1 Hz due to computational limitations. It is important to note that MoveIt supports a replanning mechanism for dynamic scenes. When obstacles are detected along the path, MoveIt triggers replanning to generate a new trajectory that avoids the obstruction.

As shown in Table VII and Fig. 13, in static scenarios, PMPPI outperforms traditional RRT-based global planners and local planners like MPPI in terms of both running time and trajectory safety (crash rate), whether obstacles are present or not. PMPPI's advantages are especially evident in dynamic obstacle avoidance scenarios, achieving a crash rate of 10.0% and a Running time of 7.381 ± 0.062 s/lap, compared to MPPI-Coll$_{ppv\theta}$, which has a crash rate of 30.0% and a Running time of 8.397 ± 1.090 s/lap. These results demonstrate PMPPI's efficiency and safety in dynamic environments. Although PMPPI has a slightly higher planning time per iteration (0.041 ± 0.000 s/iter) compared to MPPI (0.038 ± 0.000 s/iter), the difference is minimal and does not significantly impact overall performance. Both MPPI and PMPPI benefit from parallel processing capabilities, implemented using GPU and PyTorch for real-time computation.

While RRT methods are effective at generating smooth paths in static, obstacle-free environments, their computational cost is significantly higher, particularly in dynamic environments. RRT struggles with real-time obstacle avoidance, leading to higher collision rates due to delayed obstacle interaction. Failure cases, particularly those caused by occlusion or high-speed obstacles, are demonstrated in the supplementary video, highlighting the challenges faced in real-time dynamic obstacle avoidance.

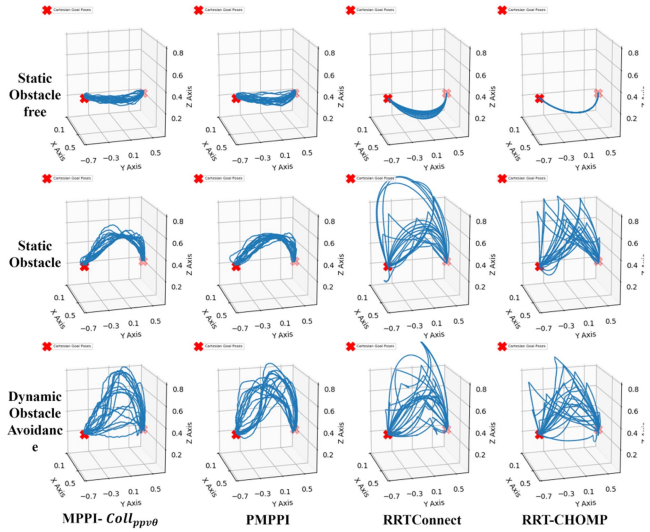


Fig. 13. EE. Trajectory visualization for different methods in three experimental scenarios.

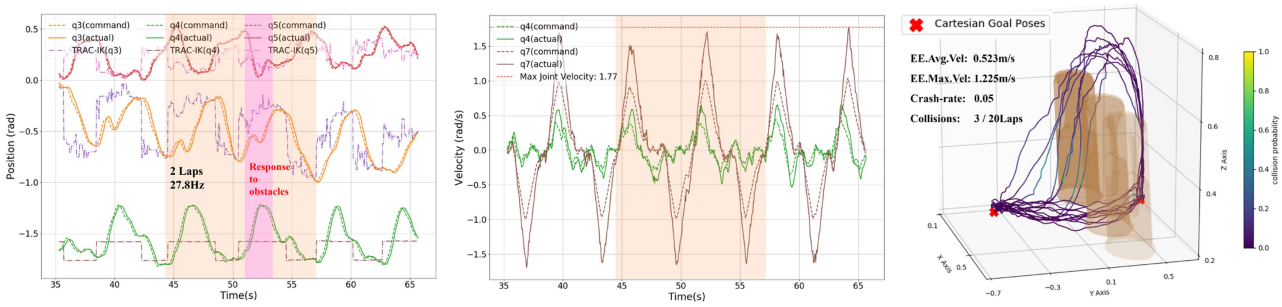
C. Dynamic Obstacle Avoidance Tests

We designed multiple dynamic obstacle avoidance experimental scenarios to comprehensively evaluate the obstacle avoidance capabilities of the PMPPI, which include: Scenario 1: Inspired by the experimental methods in [1], this scenario involved obstacles suddenly appearing in the robot's path. Scenario 2: Similar to setups in [2], obstacles moved laterally (left and right) within the robot arm's workspace.

In both scenarios, the robot executed reciprocating motions between two predefined poses for 20 laps. Performance metrics such as collision rate, path length, and computational efficiency were recorded to assess the algorithm's effectiveness.

Fig. 14 illustrates the recording of joint positions, velocities, and the EE trajectories during the running process in dynamic obstacle avoidance scenario 1, as depicted in the top row of Fig. 15. Fig. 14(a) records the joint positions, where q_{command} denotes the joint commands generated by the PMPPI, executed by a PD-controlled Franka robot, while q_{actual} represents its actual joint states. Dashed lines indicate the joint guidance data from the TRAC-IK, exhibiting a single pulse-like fluctuation over one lap of the robotic manipulator's reciprocating motion. As joint trajectories q_{actual} rapidly converge toward the TRAC-IK guided positions, noticeable fluctuations or deviations are observed, indicating the robotic manipulator dynamically responds to the sudden appearance of obstacles. The control frequency, measured at 27.8 Hz, further validates the system's responsiveness. In addition, Fig. 14(b) illustrates the recorded joint velocities, with the maximum joint velocity peaking at 1.77 rad/s. Fig. 14(c) displays the EE trajectories recorded over 20 laps. The average EE speed is measured at 0.523 m/s,

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 14. Joint positions, velocities, and EE trajectories in sudden obstacle appearance scenario (see Fig. 15, top row). (a) Joints positions. (b) Joints velocities. (c) EE trajectories.

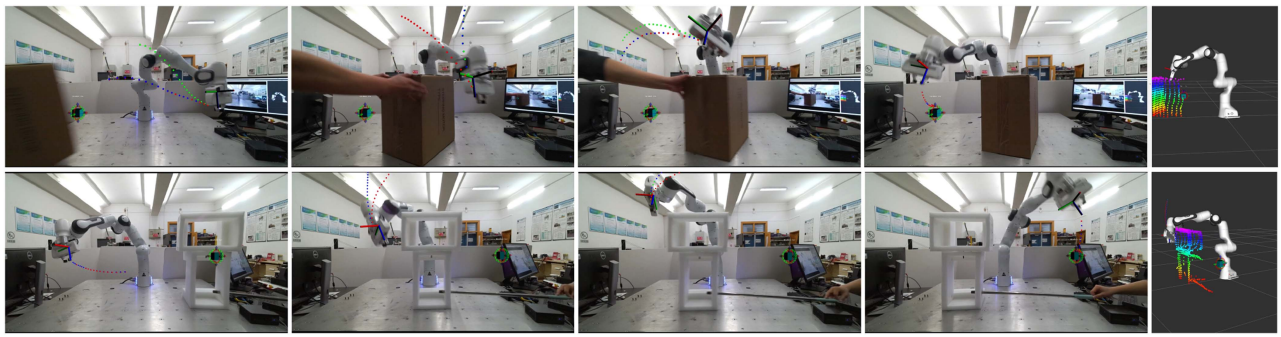


Fig. 15. PMPPI in dynamic obstacle avoidance scenarios: sudden obstacle appearance (top row) and obstacle left-right moving (bottom row). The robotic manipulator moves between two poses and arbitrary objects can serve as obstacles.

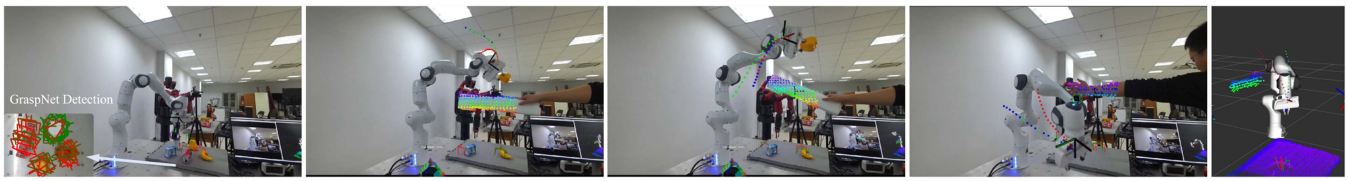


Fig. 16. PMPPI in 6-D Grasping and human–robot interaction scenarios: object grasping detection utilizes GraspNet, which provides grasp target poses from desktop point clouds.

while the maximum speed reaches 1.225 m/s. Notably, during these 20 laps, three collisions occurred, resulting in a crash rate below 5%, showing remarkable obstacle avoidance capabilities of the PMPPI algorithm. A supplementary video is available for enhanced visualization.

D. Grasping Tasks and Human–Robot Interaction

The last experiment was conducted for a pick-and-place task similar to experiment scenario depicted in paper [3]. In Fig. 16, five different objects are positioned on the desktop. The Franka robot is tasked with picking and placing all objects, while facing interference from moving obstacles or the operator. Benefiting from our purely vision-based point clouds processing approach, arbitrary objects can serve as obstacles. This algorithm naturally meets the task requirements for human–robot interaction. We utilize the Azure Kinect camera to generate point clouds of the working space, which undergo preprocessing to filter out point clouds from the robot manipulator, before being used to create the SDF map. We employ GraspNet [38], a neural network model for 6-D object grasp pose recognition, with input from desktop point clouds captured by the D435i camera, to provide grasp target poses.

Throughout the experimental process, the Franka successfully completed the grasp task of cleaning all objects on the desktop (5/5). It demonstrated rapid response and obstacle avoidance capabilities, efficiently maneuvering through various interferences caused by moving obstacles and the operator.

VIII. Discussion

A. Advantages of the PMPPI

PMPPI synergizes MME-DDP's dynamic weighting and GMR's mixture approximation with MPPI's sampling efficiency. Its multiplanner design captures a richer solution space than standard MPPI, while the Judge strategy π^* introduces a

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

novel fairness mechanism, distinguishing it from MME-DDP's planner-internal evaluations. By approximating the mixture policy, PMPPI balances computational tractability with multimodal robustness, making it well-suited for dynamic, uncertain environments like autonomous navigation.

PMPPI offers multiple advantages that make it superior than the approach of continuously tuning the weights of a single-layer MPPI. Theoretical analysis suggests that PMPPI significantly enhances exploration capabilities compared to single-layer MPPI, without compromising algorithm convergence. Experimental results and robot motion characteristics further indicate that PMPPI can adaptively adjust its mixed weights based on working conditions. Specifically, it switches to a sensitive strategy to avoid collisions when obstacles are nearby and adopts a greedy strategy to accelerate toward the target when obstacles are distant. By integrating multiple strategies, PMPPI minimizes collisions while efficiently completing tasks in the shortest possible time.

In essence, PMPPI is not simply a single planner with multiple cost functions combined during the update step. It builds upon the MPPI framework with significant innovations in sampling, cost function computation, and adaptive integration, resulting in a computationally efficient yet multimodal planning algorithm. These contributions extend the applicability of MPPI, particularly in scenarios requiring flexible strategy evaluation and adaptive decision-making.

From a theoretical perspective, PMPPI and learning-based approaches like reinforcement learning or imitation learning exhibit complementary strengths. Learning methods excel at capturing complex patterns from extensive data, potentially surpassing PMPPI in scenarios with well-defined rewards and long horizons. However, they demand significant training resources and may struggle in generalization, whereas PMPPI offers model-based, real-time adaptability without pretraining. The Judge strategy bolsters PMPPI's robustness through standardized evaluation, unlike learning approaches that rely on extensive tuning. Thus, while learning methods provide data-driven flexibility, PMPPI delivers a lightweight, interpretable solution for motion planning in dynamic environments.

In our real-world experiments, the final scenario incorporates a grasping task with human–robot interaction and dynamic obstacle avoidance, designed to illustrate the application value of reactive motion planning. This setup demonstrates PMPPI's capability to adapt to unpredictable movements-such as a human operator or moving objects-while maintaining task efficiency and safety, a critical need in collaborative robotics and industrial settings.

B. Challenges and Future Work

The proposed algorithms (the PMPPI and GVM-SDF cost functions) have been successfully deployed and validated across environments ranging from 2-D to 7-D, achieving excellent obstacle avoidance performance in all cases. This indicates the robustness and reliability of these methods. Therefore, these approaches are expected to be transferable to other serial multidegree-of-freedom manipulators without significant limitations. While our proposed methods have made significant strides in leveraging depth cameras for purely visual perception solutions, thereby expanding the application scenarios of the PMPPI, critical challenges and opportunities for future exploration remain unaddressed.

One such challenge arises from the limitations of a single-camera perspective and the occlusion caused by the robotic arm, resulting in incomplete or inaccurate point cloud data during dynamic obstacle avoidance. Addressing this issue by integrating multiple sensors or employing deep learning techniques to predict missing portions of the point cloud holds promise for enhancing the algorithm's responsiveness and robustness in real-world scenarios. As a sampling-based motion planning algorithm, PMPPI inherently produces trajectories with limited smoothness. This can lead to suboptimal paths, as observed in real-world static obstacle free task (see Table VII), where PMPPI's EE. length increased by 8.3% (2.719 m) compared to RRTConnect (2.578 m) due to jagged motions. Numerical optimization methods, such as L-BFGS [39], could refine these planned trajectories, a potential enhancement we note for future work.

Moreover, the integration of deep learning techniques for predicting obstacle movement direction based on point cloud data represents a significant opportunity for future innovation. By incorporating predictive information into motion planning algorithms like MPPI, we can potentially empower robots to anticipate and navigate around potential collisions more effectively, thereby optimizing path planning in dynamic environments.

Finally, while our current focus has been on the development and validation of the PMPPI algorithm, future research directions could explore the integration of reinforcement learning techniques. This integration has the potential to mitigate the risk of local minima and reduce computational complexity, paving the way for more efficient and adaptive robotic motion planning strategies.

IX. Conclusion

In this article, we introduce the PMPPI algorithm, enhancing exploration capabilities by concurrently running multiple MPPI planners with diverse strategies. We propose innovative cost functions tailored for the PMPPI's application in dynamic obstacle avoidance, including the GVM-SDF Cost, TRAC-IK guided cost, and sparse reward cost. Real-world experiments demonstrate the algorithm's exceptional performance, with average and maximum EE speeds of 0.523 m/s and 1.225 m/s, respectively, and a crash rate below 5%. Our approach surpasses standard MPPI methods and traditional SDF designs, offering swift and safe planning with reduced collision risks.

Appendix
Theoretical Foundations of PMPPI

This Appendix provides the theoretical foundations supporting the PMPPI algorithm presented in Section IV. We explore the multimodal extensions from MME-DDP [31], the entropy-regularized value function from MPQ [11], and the transition

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

from Gaussian mixture models (GMM) to GMR [32], which collectively inform the development of PMPPI.

A. MME-DDP: Multimodal Gaussian Policies

DDP is a powerful trajectory optimization technique, traditionally focused on generating a single optimal path by iteratively refining control sequences. However, in complex environments—such as those with multiple viable paths or dynamic obstacles—a single solution may fail to capture the full range of feasible strategies. MME-DDP [31] addresses this limitation by extending DDP into a multimodal framework, capable of optimizing several candidate trajectories simultaneously. Its core contribution lies in dynamically weighting these trajectories based on their performance, as assessed by value functions derived from DDP.

The MME-DDP policy $\hat{\pi}(u|\hat{x}_t)$, representing the optimized control distribution given current robot state $\hat{x}_t$, is formulated as a weighted combination of individual planner outputs

$$\hat{\pi}(u|\hat{x}_t) = \sum_{n=1}^{N} w^{(n)} \hat{\pi}^{(n)}(u|\hat{x}_t) \tag{31}$$

where $\hat{\pi}^{(n)}(u|\hat{x}_t) \sim \mathcal{N}(\hat{\mu}_t^{(n)}(\hat{x}_t), \hat{\Sigma}_t^{(n)}(\hat{x}_t))$ represents the Gaussian policy of the nth planner, with mean $\hat{\mu}_t^{(n)}(\hat{x}_t)$ and covariance $\hat{\Sigma}_t^{(n)}(\hat{x}_t)$ optimized through DDP iterations, and $w^{(n)}(t, \hat{x}_t)$ is the weight assigned to the nth planner, determined by its associated value function $V^{(n)}(t, \hat{x}_t)$

$$w^{(n)}(t, \hat{x}_t) = \frac{\exp\left(-\frac{1}{\lambda_{\text{ddp}}} V^{(n)}(t, \hat{x}_t)\right)}{\sum_{n'=1}^{N} \exp\left(-\frac{1}{\lambda_{\text{ddp}}} V^{(n')}(t, \hat{x}_t)\right)}. \tag{32}$$

The value function $V^{(n)}(t, \hat{x}_t)$ captures the quality of the trajectory generated by the nth planner, with $\bar{x}_t^{(n)}$ as its nominal trajectory, and is expressed as

$$\begin{aligned} V^{(n)}(t, \hat{x}_t) = \; & \bar{V}^{(n)}(t, \bar{x}_t^{(n)}) + V_H^{(n)}(t, \bar{x}_t^{(n)}) \\ & + V_x^{(n)}(t, \bar{x}_t^{(n)})^\top \delta\hat{x}_t + \frac{1}{2} \delta\hat{x}_t^\top V_{xx}^{(n)}(t, \bar{x}_t^{(n)}) \delta\hat{x}_t \end{aligned} \tag{33}$$

where $\bar{V}^{(n)}(t, \bar{x}_t^{(n)})$ represents the nominal scalar cost of the nth trajectory $\bar{x}_t^{(n)}$, $V_H^{(n)}(t, \bar{x}_t^{(n)})$ is the entropy regularization term to encourage exploration, and $V_x^{(n)}(t, \bar{x}_t^{(n)})$ and $V_{xx}^{(n)}(t, \bar{x}_t^{(n)})$ are the gradient and Hessian of the value function, respectively, used to capture local trajectory variations. Here, $\delta\hat{x}_t = \hat{x}_t - \bar{x}_t^{(n)}$ denotes the deviation from the nominal trajectory $\bar{x}_t^{(n)}$. This formulation allows MME-DDP to dynamically optimize multimodal policies, leveraging a tradeoff between exploitation of high-performing trajectories and exploration of alternative solutions.

The resulting $\hat{\pi}(u|\hat{x}_t)$ is a Gaussian mixture distribution, reflecting the multimodal nature of the solution space. Each planner may represent a distinct strategy-such as aggressive goal-seeking or cautious obstacle avoidance-allowing MME-DDP to adaptively prioritize options based on the current state $\hat{x}_t$ and time t.

B. MPQ: Entropy-Regularized Value Function

The MPQ [11] framework extends maximum entropy principles to MPPI, enabling robust trajectory evaluation under uncertainty. The value function $V^\pi(\hat{x}_t)$ evaluates the quality of trajectory distributions under the stochastic policy π, a probability distribution over control inputs. The value function can be formulated as

$$\begin{aligned} V^\pi(\hat{x}_t) = -\lambda_{\text{mpq}} \log \mathbb{E}_\pi \Bigg[\exp\Bigg(-\frac{1}{\lambda_{\text{mpq}}} \Bigg(& \sum_{h=0}^{H-1} c(\hat{x}_{t+h}, u_{t+h}) \\ & + Q^\pi(\hat{x}_{t+H}, u_{t+H}) \Bigg) \Bigg) \Bigg] \end{aligned} \tag{34}$$

where $c(\hat{x}_t, u_t)$ represents the instantaneous cost at state $\hat{x}_t$ and control u_t, $Q^\pi(\hat{x}_{t+H}, u_{t+H})$ is the terminal value function capturing the cost of the trajectory's endpoint, H is the planning horizon, and λ_{mpq} controls the sensitivity of the value function to trajectory cost variations. Entropy regularization enhances trajectory diversity, ensuring robust performance in dynamic environments.

C. Transition From GMM to GMR

While MME-DDP optimizes multimodal policies through control theory, real-world robotic systems often benefit from probabilistic models that capture uncertainty in state-control relationships. In trajectory planning, the joint distribution $P(\hat{x}_t, u)$ (modeled using GMM) captures the probabilistic relationship between the robot's state $\hat{x}_t$ and the control u. However, practical trajectory optimization and control require the conditional distribution $P(u|\hat{x}_t)$, which enables the generation of optimal control actions u for a given state $\hat{x}_t$. GMR [40], [41], [42] provides the theoretical framework for this transition. By leveraging the statistical properties of GMM, GMR extracts $P(u|\hat{x}_t)$ from the joint distribution $P(\hat{x}_t, u)$, enabling the alignment of control distributions with the current robot state. Importantly, GMR approximates the multimodal conditional distribution as a single Gaussian, facilitating its application to adaptive trajectory planning and dynamic control.

Given the joint distribution $P(\hat{x}_t, u)$ of states $\hat{x}_t$ and controls u modeled as a GMM

$$P(\hat{x}_t, u) = \sum_{n=1}^{N} w^{(n)} \mathcal{N}\left(\begin{bmatrix} \hat{x}_t \\ u \end{bmatrix} \middle| \hat{\mu}^{(n)}, \hat{\Sigma}^{(n)} \right) \tag{35}$$

where $w^{(n)}$ is the weight of the nth Gaussian component, $\hat{\mu}^{(n)} = \begin{bmatrix} \mu_x^{(n)} \\ \mu_u^{(n)} \end{bmatrix}$ is the joint mean vector of states and controls, and $\hat{\Sigma}^{(n)} = \begin{bmatrix} \Sigma_{xx}^{(n)} & \Sigma_{xu}^{(n)} \\ \Sigma_{ux}^{(n)} & \Sigma_{uu}^{(n)} \end{bmatrix}$ is the joint covariance matrix. This

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

mixture allows GMM to represent complex, multimodal relationships, for example, multiple control options for a given state in a cluttered environment.

The overall conditional distribution $P(u|\hat{x}_t)$ is reconstructed as a weighted sum of the conditional distributions for each component using GMR

$$P(u|\hat{x}_t) = \sum_{n=1}^{N} h^{(n)}(\hat{x}_t)\mathcal{N}\left(u\middle|\mu_u^{(n)}(\hat{x}_t), \Sigma_u^{(n)}\right) \tag{36}$$

where the conditional weight, reflecting the likelihood of $\hat{x}_t$ belonging to the nth component, is defined as

$$h^{(n)}(\hat{x}_t) = \frac{w^{(n)}\mathcal{N}\left(\hat{x}_t\middle|\mu_x^{(n)}, \Sigma_{xx}^{(n)}\right)}{\sum_{i=1}^{N} w^{(i)}\mathcal{N}\left(\hat{x}_t\middle|\mu_x^{(i)}, \Sigma_{xx}^{(i)}\right)} \tag{37}$$

while the conditional mean and covariance are given by

$$\begin{aligned}\hat{\mu}_u^{(n)}(\hat{x}_t) &= \mu_u^{(n)} + \Sigma_{ux}^{(n)}(\Sigma_{xx}^{(n)})^{-1}(\hat{x}_t - \mu_x^{(n)}) \\ \Sigma_u^{(n)} &= \Sigma_{uu}^{(n)} - \Sigma_{ux}^{(n)}(\Sigma_{xx}^{(n)})^{-1}\Sigma_{xu}^{(n)}.\end{aligned} \tag{38}$$

To simplify this multimodal conditional distribution into a single Gaussian distribution, GMR [32] computes the weighted mean and covariance as

$$\begin{aligned}\hat{\mu}_u(\hat{x}_t) &= \sum_{n=1}^{N} h^{(n)}(\hat{x}_t)\hat{\mu}_u^{(n)}(\hat{x}_t) \\ \hat{\Sigma}_u(\hat{x}_t) &= \sum_{n=1}^{N} h^{(n)}(\hat{x}_t)\left(\Sigma_u^{(n)} + \mu_u^{(n)}(\hat{x}_t)\mu_u^{(n)}(\hat{x}_t)^\top\right) \\ &\quad - \hat{\mu}_u(\hat{x}_t)\hat{\mu}_u(\hat{x}_t)^\top.\end{aligned} \tag{39}$$

Through GMR, the multimodal distribution $P(\hat{x}_t, u)$ is approximated as a single Gaussian

$$P(u|\hat{x}_t) \approx \mathcal{N}(\hat{\mu}_u, \hat{\Sigma}_u) \tag{40}$$

where $\hat{\mu}_u(\hat{x}_t)$ represents the optimal probabilistic control action given the current state. $\hat{\Sigma}_u(\hat{x}_t)$ captures the uncertainty around the control action, providing confidence bounds for planning.

By extracting $P(u|\hat{x}_t)$, GMR provides the capability to predict control distributions that align with the current robot state $\hat{x}_t$, forming a probabilistic reference trajectory. This is essential for trajectory optimization and adaptive control in dynamic environments. This approximation simplifies computation while preserving the essence of the multimodal input, a technique we adapt in PMPPI to balance complexity and efficiency. GMR's transition from a statistical mixture to a practical control distribution bridges the gap between data-driven modeling and real-time decision-making.

MME-DDP's optimization of multimodal policies and GMR's statistical handling of mixture distributions provide complementary strengths: performance-driven weighting and probabilistic simplification. PMPPI marries these ideas with MPPI's sampling-based control, introducing a multiplanner framework that enhances adaptability and fairness in trajectory planning.

REFERENCES

[1] A. Fishman, A. Murali, C. Eppner, B. Peele, B. Boots, and D. Fox, "Motion policy networks," in *Proc. Conf. Robot Learn.*, 2023, pp. 967–977.

[2] J. Yamada, C.-M. Hung, J. Collins, I. Havoutis, and I. Posner, "Leveraging scene embeddings for gradient-based motion planning in latent space," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 5674–5680.

[3] V. Vasilopoulos, S. Garg, P. Piacenza, J. Huh, and V. Isler, "RAMP: Hierarchical reactive motion planning for manipulation tasks using implicit signed distance functions," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 10551–10558.

[4] S. M. LaValle, *Planning Algorithms*, Cambridge, U.K., Cambridge Univ. Press, 2006.

[5] C. K. Fourie, N. Figueroa, and J. A. Shah, "On-manifold strategies for reactive dynamical system modulation with nonconvex obstacles," *IEEE Trans. Robot.*, vol. 40, pp. 2390–2409, Mar. 2024, doi: 10.1109/TRO.2024.3378179.

[6] A. Heins and A. P. Schoellig, "Keep it upright: Model predictive control for nonprehensile object transportation with obstacle avoidance on a mobile manipulator," *IEEE Robot. Autom. Lett.*, vol. 8, no. 12, pp. 7986–7993, Dec. 2023.

[7] J. Nubert, J. Köhler, V. Berenz, F. Allgöwer, and S. Trimpe, "Safe and fast tracking on a robot manipulator: Robust MPC and neural network control," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3050–3057, Apr. 2020.

[8] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1603–1622, Dec. 2018.

[9] M. Bhardwaj et al., "STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. Conf. Robot Learn.*, 2022, pp. 750–759.

[10] W. Yang et al., "Model predictive control for fluid human-to-robot handovers," in *Proc. Int. Conf. Robot. Autom.*, 2022, pp. 6956–6962, doi: 10.1109/ICRA46639.2022.9812109.

[11] M. Bhardwaj, A. Handa, D. Fox, and B. Boots, "Information theoretic model predictive Q-learning," in *Proc. Learn. Dyn. Control*, 2020, pp. 840–850.

[12] J. Jankowski, L. Brudermüller, N. Hawes, and S. Calinon, "VP-STO: Via-point-based stochastic trajectory optimization for reactive robot behavior," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 10125–10131.

[13] J. Sacks and B. Boots, "Learning sampling distributions for model predictive control," in *Proc. Conf. Robot Learn.*, 2023, pp. 1733–1742.

[14] J. Yin, Z. Zhang, and P. Tsiotras, "Risk-aware model predictive path integral control using conditional value-at-risk," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 7937–7943.

[15] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, "Shield model predictive path integral: A computationally efficient robust MPC method using control barrier functions," *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7106–7113, Nov. 2023.

[16] E. Gilbert, D. Johnson, and S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, Apr. 1988, doi: 10.1109/56.2083.

[17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2012, pp. 3859–3866, doi: 10.1109/ICRA.2012.6225337.

[18] S. Laine and T. Karras, "Efficient sparse voxel octrees–analysis, extensions, and implementation," *NVIDIA Corp.*, vol. 2, no. 6, pp. 1–30, 2010.

[19] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, "3D U-Net: Learning dense volumetric segmentation from sparse annotation," in *Proc. 19th Int. Conf. Med. Image Comput. Comput.-Assisted Interven.*, Athens, Greece, 2016, pp. 424–432.

[20] Y. Zhou and O. Tuzel, "VoxelNET: End-to-end learning for point cloud based 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4490–4499.

[21] Z. Wu et al., "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1912–1920.

[22] J. Pan, S. Chitta, and D. Manocha, "Probabilistic collision detection between noisy point clouds using robust classification," in *Proc. Robot. Research: 15th Int. Symp.*, 2017, pp. 77–94.

[23] M. Danielczuk, A. Mousavian, C. Eppner, and D. Fox, "Object rearrangement using learned implicit collision functions," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 6010–6017.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.

[24] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 4569–4574.
[25] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2009, pp. 489–494.
[26] M. Koptev, N. Figueroa, and A. Billard, "Neural joint space implicit signed distance functions for reactive robot manipulator control," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 480–487, Feb. 2023.
[27] P. Liu, K. Zhang, D. Tateo, S. Jauhri, J. Peters, and G. Chalvatzaki, "Regularized deep signed distance fields for reactive motion generation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 6673–6680.
[28] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *Proc. IEEE-RAS 15th Int. Conf. Humanoid Robots*, 2015, pp. 928–935, doi: 10.1109/HUMANOIDS.2015.7363472.
[29] A. Fishman, C. Paxton, W. Yang, D. Fox, B. Boots, and N. Ratliff, "Collaborative interaction models for optimized human-robot teamwork," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 11221–11228,doi: 10.1109/IROS45743.2020.9341369.
[30] J. Yin, Z. Zhang, E. Theodorou, and P. Tsiotras, "Trajectory distribution control for model predictive path integral control using covariance steering," in *Proc. Int. Conf. Robot. Autom.*, 2022, pp. 1478–1484.
[31] O. So, Z. Wang, and E. A. Theodorou, "Maximum entropy differential dynamic programming," in *Proc. Int. Conf. Robot. Autom.*, pp. 3422–3428, 2022.
[32] Y. Huang, F. J. Abu-Dakka, J. Silvério, and D. G. Caldwell, "Toward orientation learning and adaptation in Cartesian space," *IEEE Trans. Robot.*, vol. 37, no. 1, pp. 82–98, Feb. 2021.
[33] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2014, pp. 709–715.
[34] V. Makoviychuk et al., "ISAAC gym: High performance GPU-based physics simulation for robot learning," 2021, *arXiv:2108.10470*.
[35] J. Haviland and P. Corke, "NEO: A novel expeditious optimisation algorithm for reactive motion control of manipulators," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1043–1050, Apr. 2021, doi: 10.1109/LRA.2021.3056060.
[36] T. Kröger and F. M. Wahl, "Online trajectory generation: Basic concepts for instantaneous reactions to unforeseen events," *IEEE Trans. Robot.*, vol. 26, no. 1, pp. 94–111, Feb. 2010, doi: 10.1109/TRO.2009.2035744.
[37] Peci1, "Filters the robot's body out of point clouds and laser scans," 2023. [Online]. Available: https://github.com/peci1/robot_body_filter
[38] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "Graspnet-1billion: A large-scale benchmark for general object grasping," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11444–11453.
[39] B. Sundaralingam et al., "CuRobo: Parallelized collision-free robot motion generation," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 8112–8119. doi: 10.1109/ICRA48891.2023.10160765.
[40] S. Calinon, "A tutorial on task-parameterized movement learning and retrieval," *Intell. Service Robot.*, vol. 9, pp. 1–29, 2016.
[41] D. A. Cohn, Z. Ghahramani, and M. I. Jordan, "Active learning with statistical models," *J. Artif. Intell. Res.*, vol. 4, pp. 129–145, 1996.
[42] Y. Huang, L. Rozo, J. Silvério, and D. G. Caldwell, "Kernelized movement primitives," *Int. J. Robot. Res.*, vol. 38, no. 7, pp. 833–852, 2019.



Lelai Zhou (Member, IEEE) received the B.Eng. degree in mechanical engineering and automation from the University of Science and Technology, Beijing, China, in 2006, the M.Eng. degree in mechanical design and theory from Beihang University, Beijing, in 2009, and the Ph.D. degree in mechanical engineering from Aalborg University, Aalborg, Denmark, in 2012.

He is currently a Professor with the School of Control Science and Engineering, Shandong University, Jinan, China. He has authored or coauthored more than 70 journal and conference articles. His main researches are robotic arm, quadruped robot, human–robot interaction, and multirobot collaboration.

Dr. Zhou was a recipient of the Shandong Provincial Natural Science Foundation for Distinguished Young Scholars in 2022.



Zhengmao Li received the B.Eng. degree in automation engineering from Nanjing Tech University, Nanjing, China, in 2022. He is currently working toward the M.Eng. degree in control science and engineering with the School of Control Science and Engineering, Shandong University, Jinan, China.

His research interests include dynamic obstacle avoidance for robotic arms and human–robot collaboration.



Yibin Li (Member, IEEE) received the B.Eng. degree in automation from Tianjin University, Tianjin, China, in 1982, the M.Eng. degree in electrical automation from the Shandong University of Science and Technology, Qingdao, China, in 1990, and the Ph.D. degree in automation from Tianjin University, Beijing, China, in 2008.

From 1982 to 2003, he was with the Shandong University of Science and Technology. Since 2003, he has been the Director of the Center for Robotics, Shandong University, Jinan, China. His research interests include robotics, intelligent control theories, and computer control systems.



Shaoping Bai (Senior Member, IEEE) received the Ph.D. degree in mechanical and production engineering from Nanyang Technological University, Singapore, in 2001.

He is currently a Full Professor with the Department of Materials and Production, Aalborg University, Aalborg, Denmark. His research interests include exoskeletons, wearable sensors, parallel manipulators, and walking robots.

Dr. Bai is a member of ASME, a senior member of IEEE Robotics and Automation Society, and elected member of IFToMM Executive Council. He is currently an Associate Editor for the ASME Letters in Translational Robotics, Robotica and Wearable Technologies.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 20,2026 at 13:55:20 UTC from IEEE Xplore. Restrictions apply.